Transportation Research Part B 202 (2025) 103335

Contents lists available at ScienceDirect

Transportation Research Part B

journal homepage: www.elsevier.com/locate/trb

A dynamic drone routing problem with uncertain demand and energy consumption

Guilherme O. Chagas [a,*], Leandro C. Coelho [a,b], Demetrio Laganà [c], Patrizia Beraldi [d]

[a] CIRRELT and Faculté des Sciences de l'Administration, Université Laval, Québec, Canada
[b] Canada Research Chair in Integrated Logistics, Université Laval, Québec, Canada
[c] CIRRELT and Department of Mechanical, Energy and Management Engineering (DIMEG), University of Calabria, Via P. Bucci Cubo 41C, 87036, Arcavacata di Rende, (CS), Italy
[d] Department of Mechanical, Energy and Management Engineering (DIMEG), University of Calabria, Via P. Bucci Cubo 41C, 87036, Arcavacata di Rende, (CS), Italy

ARTICLE INFO

Keywords:
Combinatorial optimization
Drone routing
Markov decision process
Cost function approximation
Uncertain demand
Uncertain energy consumption

ABSTRACT

This work addresses a drone routing problem with an identical fleet performing same-day deliveries in a dynamic and uncertain environment. We model the problem as a Markov Decision Process to capture the stochastic nature of customer demand and the uncertainty in energy consumption due to varying payloads and weather conditions. To tackle this problem, we propose an approximate dynamic algorithm that integrates routing planning, drone usage, and battery management. Uncertainty in energy consumption is dealt with the chance constraints ensuring that drone trips are completed safely, preventing premature returns to the depot. The proposed approach features a cost function approximation policy that accounts for a restricted number of trips to be assigned to drones. This ensures that the drones are ready at the depot to fulfill new requests that may arise during the day. Extensive computational experiments on 300 instances validate the effectiveness of our method, demonstrating its superiority over a myopic strategy, a policy function approximation approach, and an oracle method, thus highlighting its potential for practical applications.

1. Introduction

Drone delivery can redefine the shipping market as drones offer a promising alternative to traditional delivery methods, especially in congested urban areas or locations that are difficult to reach by conventional vehicles. Moreover, they can contribute to sustainable development as electric-powered drones produce fewer emissions than traditional delivery vehicles (Garg et al., 2023). Large distributors, like Amazon, have already tried these innovative ways to deliver goods. These services have also emerged within the grocery and restaurant domain, e.g., Uber Eats Drone Delivery and KFC Drone fast food delivery focusing on delivering food within tight deadlines and time windows. Aside from commercial use, drones can deliver relief items in humanitarian logistics and refill medicines in healthcare projects.

Ongoing improvements in drone technology, such as increased payload capacity, extended range, and improved safety features, continue to expand the potential of drone delivery, posing new challenges in optimizing logistic operations. In particular, the need

* Corresponding author.
E-mail addresses: guilherme.oliveira-chagas.1@ulaval.ca (G.O. Chagas), leandro.coelho@fsa.ulaval.ca (L.C. Coelho), demetrio.lagana@unical.it (D. Laganà), patrizia.beraldi@unical.it (P. Beraldi).

https://doi.org/10.1016/j.trb.2025.103335
Received 29 August 2024; Received in revised form 18 September 2025; Accepted 29 September 2025
Available online 21 October 2025

to consider energy constraints, stochastic travel times, and weather-dependent variability in drone behavior has been identified as a key research direction (Poikonen and Campbell, 2020). In this paper, we address the problem of optimizing a daily delivery service by exploiting a fleet of identical drones under demand and energy consumption uncertainty. This entails designing a system that dynamically creates trips and optimizes their assignment to drones, considering the evolving delivery requirements throughout the day.

In optimizing drone operations, batteries represent the most critical resource. Most medium-sized civilian drones are powered by lithium-ion batteries that can guarantee a limited airborne time. Thus, drones should make multiple trips with frequent stops at the depot for battery swaps, i.e., replacing the discharged battery with a fully charged one before performing another trip. Properly addressing battery usage and fully exploiting their range ensures an efficient system design. In our approach, we build trips serving multiple requests, each delivering a small package to a customer location. In the rest of the paper, we use the terms *request*, *delivery*, *customer*, *delivery request* or *customer request* interchangeably. Likewise, as the *depot* is used as *charging station*, these terms are used interchangeably as well.

The number of deliveries on a trip depends on the total payload limited by the drone capacity and energy consumption. Our approach assumes that consumption is a function of the payload and flight time. Considering load-dependent consumption is essential for accurate energy estimates since heavier loads require more energy. In addition, drones are more sensitive to weather conditions than traditional vehicles. For example, wind can influence the drone's speed and, thus, the flight time and range. To account for uncertainty, we assume that the energy consumption is random, and we employ the paradigm of chance constraints to build safe trips, avoiding routing the drones back to the depot prematurely and delaying the delivery service.

We model our problem setting as a Markov decision process (MDP) to represent the dynamic stochastic nature of the request arrival. Moreover, since we consider time-sensitive requests, we allow customers to be served after their soft deadline. The objective of our problem is to define routing plans that maximize the number of customers served while minimizing the total expected cost, which is the sum of the expected routing cost and the cost of the expected lateness to make deliveries. Finally, dynamic drone dispatching is carried out at the depot to ensure equal battery usage.

Our work contributes to the literature on drone delivery problems defining a decision approach that accounts for the dynamic stochastic process of request arrival, the uncertainty affecting energy consumption, and the deterioration of batteries. More specifically, our paper makes the following contributions to existing literature:

- From a modeling perspective, we propose a decision-making framework that integrates routing decisions with ground service operations, specifically focusing on battery charging and assignment to drones. This holistic approach captures the interdependencies between in-flight and on-ground activities;
- In a novel approach, we address two significant challenges often treated separately in the literature. In particular, we propose an MDP model accounting for the dynamic and stochastic nature of the arrival process, whereas uncertainty in energy consumption is considered by applying the paradigm of chance constraints;
- Methodologically, we present a tailored cost function approximation (CFA) policy to support decision-making. This policy is constructed by decomposing the overall decision process into a sequence of straightforward actions, where each action is determined by solving a mathematical formulation whose optimal solution serves as the input for the subsequent stage. To address future service requests when making current decisions, we introduce a calibrated threshold that limits the maximum number of trips assigned to each drone at each decision epoch. Additionally, we develop a policy function approximation (PFA) and an oracle method to evaluate the performance of our CFA;
- The paper provides extensive computational experiments on instances derived from the literature. We compare our CFA against a myopic policy, the PFA approach, and the oracle method. The results consistently demonstrate the superior performance and effectiveness of our approach over traditional, myopic strategies.

In our contribution, we focus on a same-day delivery problem, and we assume that the delivery requests are not known in advance, but they arrive randomly during the day. Each request is associated with a (soft) deadline, limiting the time between its realization and the service time. This temporal dimension, reflecting the time sensitivity in the fulfillment of the customer requests, makes our framework general and applicable in other contexts, such as in emergency medical services, where requests with tight due dates require immediate attention and have a higher priority in the route planning phase.

The paper is organized as follows. In Section 2, we review the relevant literature and position our work within the existing research. Section 3 describes the problem, whereas Section 4 presents the mathematical modeling of the problem as an MDP. The solution approach of our CFA method is described in Section 5, where its steps and actions are detailed in Section 6. Section 7 describes the PFA which we use to benchmark our CFA policy. In Section 8, we outline the data sets and present the results of computational tests of our solution approach. Finally, Section 9 provides our conclusions and discusses future work.

2. Literature review

Delivery problems involving drones are typically classified into pure and combined truck-drone problems. In the first case, drones represent the sole vehicles operated by the service provider, whereas in the second case, drones cooperate with ground vehicles to complete the service to customers. Routing problems of the second class have been extensively studied since the introduction of the first problem combining a truck and a drone by Murray and Chu (2015). Later on, different variants of the problem have been investigated. We refer interested readers to Macrina et al. (2020), where the authors provide a drone problem taxonomy and classification according to the different cooperation models.

The drone-only problem, often framed as a variant of the vehicle routing problem (VRP), has received comparatively less attention. Usually, this problem is studied as a simple extension of VRP models, and more specialized modeling is required to capture the issues that arise from the drone routing context (Poikonen and Campbell, 2020). Dorling et al. (2017) proposed a multi-trip vehicle routing formulation in which a single drone at a depot executes multiple trips to satisfy the delivery requests. A notable contribution was examining the relationship between the drone's energy consumption and the total payload, including the weight of the battery. Addressing this relationship led to the formulation of a challenging mixed-integer programming problem, which was solved using a simulated annealing heuristic algorithm.

Battery recharging plays a relevant role when planning drone routing. Ermağan et al. (2022) addressed a drone routing problem that explicitly accounts for battery recharging. The authors proposed a learning-based approach that incorporates recharging stops in route planning. They formulated the problem as an integer program and used a column generation approach to generate solutions, which were used to train the learning-based heuristic. This heuristic generates routes plans by starting from a Hamiltonian path and including recharge stops as needed.

Coelho et al. (2017) studied a multi-objective routing problem where charging stations are used to extend the limited range of drones. The proposed problem has seven different objective functions, one of which is the minimization of the energy required by the drone batteries. However, the energy required is estimated using a simple function that depends on the drone's speed instead of a realistic consumption model.

Cheng et al. (2020) extended the multi-trip formulation proposed in Dorling et al. (2017) by modeling the energy consumption as a nonlinear function of payload and travel distance. The problem is solved by a specialized branch-and-cut algorithm where logical cuts and subgradient cuts are introduced to tackle the nonlinear (convex) energy consumption function. We also refer the reader to Zhang et al. (2021) for a review, classification, and assessment of the main drone energy consumption models proposed in the scientific literature.

Particularly, on drone routing under stochastic and dynamic environments, the problem has been driven by the growing relevance of unmanned aerial vehicles (UAVs), both in commercial and humanitarian logistics. Additionally, operational complexity arises from real-world uncertainties, including demand variability, weather conditions, and battery limitations.

The body of literature on stochastic and dynamic vehicle routing is extensive, yet its extension to UAVs is relatively recent development. Zhao et al. (2022) introduced a robust formulation of the traveling salesman problem with multiple drones, considering uncertain navigation environments, notably weather-induced disturbances. Their robust optimization framework exploits a scenario-based approach to manage path uncertainty, aligning with the need to safeguard operational feasibility in adverse conditions.

In humanitarian logistics, Faiz et al. (2024) developed a two-echelon vehicle-UAV routing model with robust optimization, explicitly capturing uncertainty in drone launch and delivery operations during disaster relief efforts. Yin et al. (2023) advanced this perspective by integrating uncertain demands and truck travel times in a hybrid truck-drone delivery framework, also aimed at post-disaster logistics. Yang et al. (2023) proposed a robust truck-drone coordination model that incorporates road traffic uncertainty and delivery reliability using a stochastic dominance approach.

These contributions highlight the increasing focus on robustness and uncertainty in drone-enabled delivery systems. However, most of these models remain static or scenario-based, offering limited adaptability to dynamically evolving demand and energy consumption profiles over a same-day planning horizon.

A few papers consider uncertainty in energy consumption in the drone delivery problems. Among them, we cite Pugliese et al. (2021), where the authors acknowledged the importance of considering uncertainty in energy consumption and model the drone and truck problem with the robust optimization paradigm. In Cheng et al. (2024), the authors analyze the influence of weather conditions (mainly wind) in drone flight and propose a two-period data-driven adaptive distributionally robust approach where wind observation data are used to improve scheduling decisions. Specifically, the scheduling decisions for the fleet of drones are made in the morning, with the provision for different delivery schedules in the afternoon that adapt to updated weather information available by midday. Unlike our approach, the delivery requests are assumed to be known in advance.

In our contribution, we explicitly deal with uncertainty in energy consumption. As in Dorling et al. (2017), we assume that the energy consumption can be expressed as a function of the travel time and the load carried out by a drone. Uncertainty in the weather conditions affects travel time and, as a consequence, energy consumption. Moreover, considering load-dependent energy consumption makes the delivery problem even more challenging. As packages are delivered, the drone gets lighter, and energy consumption changes. Thus, accounting for load variation becomes essential for accurately estimating energy consumption. This also impacts the assignment of the delivery requests to the available drones to reduce the overall energy consumption during the delivery process. Unlike the contributions mentioned above, we employ the chance constraint paradigm (Ruszczyński and Shapiro, 2003) to build trips that satisfy the energy requirement with a high probability level, thus avoiding drones being routed back to the depot before completing the assigned trip.

Another stream of literature related to our problem is the stochastic dynamic VRP (SDVRP). Unlike static (deterministic) problems, where all information is assumed to be known at the time of decision-making, in a dynamic setting, routing actions are taken under incomplete information. The new logistic business models prioritize instant gratification and real-time fulfillment and are the main drivers of this paradigm shift. Applications range from ride-hailing services (Gao et al., 2024) to restaurant delivery (Ulmer et al., 2021), as well as emergency repair or health care services (van Steenbergen et al., 2023). Uncertainty is typically related to the arrival of new requests (see, e.g., Ulmer et al., 2019 and Zhang and Woensel, 2023) and changes in the fleet size, for example, by the inclusion of crowdsourced drivers and their behavior (Ulmer and Savelsbergh, 2020).

SDVRPs are generally modeled as MDPs, where a sequence of routing decisions is defined in reaction and anticipation of newly revealed stochastic information. Interested readers are referred to the recent review of Soeffker et al. (2022), where the authors analyze different contributions on SDVRP in the light of prescriptive analytics, focusing on integrating information models into decision models via computational methods. Also related to the problem considered in this work are the contributions of Chen et al. (2022, 2023), where the authors analyze a same-day delivery problem with stochastic customer requests. In these studies, the fleet comprises conventional vehicles and drones.

Another work related to our study is that of Ulmer and Thomas (2018), where the authors investigate the impact of using a heterogeneous fleet for delivery service in a dynamic routing problem for the first time. Uncertainty is related to customer requests being revealed throughout the day. Indeed, until the request is made, no temporal and spatial information is available. When a new request occurs, the service provider needs to decide whether the new request is accepted or not. If service is offered, it must be conducted before a deadline, within a predefined time after the request is placed, and the provider should further decide if the delivery must be performed by a vehicle or a drone. Modifications should be made to the planned trips to accommodate new requests and maximize the expected number of deliveries per day.

Similar to these contributions, this work assumes dynamic uncertainty in customer requests, but we only consider a fleet of drones that may serve more than one customer per trip. More importantly, we deal with ground services operations that affect the definition of routing plans. Moreover, uncertainty in the energy consumption affecting the routing trip design is explicitly accounted for by integrating probabilistic constraints. Finally, the problem is tackled using an algorithm that iteratively and dynamically plans both the routes and drone deployment.

From a methodological standpoint, our approach builds on the MDP framework to dynamically adapt drone deployment and routing decisions in response to new information. While MDP-based formulations are well-established in stochastic dynamic routing (e.g., the work of Ulmer and Thomas, 2018), their application to UAV operations remains scarce. Our use of CFA aligns with policy-based methods developed in dynamic programming, enabling scalable decision-making in large state-action spaces. Notably, CFA policies have shown strong empirical performance in related problems such as dynamic vehicle routing and meal delivery (Ulmer, 2017; Cazenave et al., 2021; Stoia et al., 2025), yet their adaptation to multi-trip, energy-constrained drone operations in stochastic settings remains novel.

This work contributes to bridging these gaps by combining within a dynamic MDP-based model a CFA policy structure and chance constraints for energy safety. In doing so, it simultaneously accounts for demand uncertainty, battery management, and stochastic energy consumption in real-time - a combination not comprehensively addressed in existing literature.

3. Problem description

This section formally describes the drone routing problem accounting for ground service operations. We focus on the same-day delivery problem and assume that a delivery company randomly receives requests during the day. The working day is discretized and represented by the time horizon $\mathcal{T} = \{0, 1, \dots, T\}$. Each request r is associated with a tuple of information: the delivery location (i_r), the corresponding load (q_r), and a soft deadline ($l_r \in \mathcal{T}$) representing a time within which the request should be fulfilled after it is placed. In addition, a service time (η), assumed to be the same for all deliveries, is also considered.

Delivery requests come from an area that can be reached by drones based at a depot. The service area is represented by a complete undirected graph $\mathcal{G} = (\mathcal{N}, \mathcal{A})$, where the set $\mathcal{N}$ includes the depot, identified by node i_0, and the vertices associated with the customer locations $\mathcal{N}' = \{i_1, i_2, \dots, i_m\}$. The set of edges $\mathcal{A}$ includes all possible links between pairs of vertices, and each edge $\{i_r, i_p\}$ is associated with a travel distance (cost) denoted as $d_{i_r i_p}$, where the triangle inequality holds.

Delivery service is provided by a fleet of homogeneous drones identified by the set $\Delta = \{\delta_1, \delta_2, \dots, \delta_n\}$, each with the same capacity Q and self-weight v, including the weight of both the drone itself and its battery. The batteries have limited energy capacity, thus allowing for limited airborne time. This implies that drones must repeatedly return to the depot to swap the battery, i.e., replacing the discharged battery with a fully charged one, before loading parcels for the next deliveries.

A limited number of batteries is assumed to be available at the depot. These batteries and those equipped in the drones compose the set of batteries $\mathcal{B}$, where $|\mathcal{B}| = b$. Unlike other contributions, e.g., Dorling et al. (2017), we do not assume that the delivery company has enough fully charged batteries to meet the drone's energy needs for the day. Swapped-out batteries are charged at the depot while drones operate and are swapped into other drones in need once fully charged. We denote by ρ the average time needed to perform the swapping operation and to equip drones. The recharging time depends on the residual energy of each used battery.

It is trivial to observe that accounting for charging operations significantly increases the complexity of the drone routing problem, as it requires careful coordination between routing decisions and ground service activities. Batteries typically endure only a few hundred charge-discharge cycles over their lifetime (French, 2020), making battery management a critical factor in operational sustainability. To prolong battery life and maintain system reliability, it is essential to distribute battery usage uniformly, ensuring balanced aging across all units. In our approach, battery assignment to drones explicitly considers this balance while defining trips that serve multiple requests within the constraints of drone payload capacity and limited energy autonomy.

Given these operational constraints, accurately modeling the energy consumption of drones during flight is fundamental. Multiple studies have shown that drone energy consumption increases approximately linearly with payload. For instance, Dorling et al. (2017) formulated an energy model for multirotor drones demonstrating this linear relationship between energy use and payload plus battery weight. Likewise, Muli et al. (2022) compared different energy models and confirmed the linear dependency of energy consumption on payload for steady flight conditions. This linear relationship provides a realistic yet tractable foundation for energy modeling in our routing and scheduling context.

Accordingly, following Dorling et al. (2017), the energy $e_{i_r i_p}$ required for a drone to travel between two locations i_r and i_p is computed as $e_{i_r i_p}(q_{i_r i_p}, \xi_{i_r i_p}) = \frac{d_{i_r i_p}}{\xi_{i_r i_p}} \left((v + q_{i_r i_p})^{\frac{3}{2}} \sqrt{\frac{g^3}{2\rho z m}} \right)$. In this equation, $q_{i_r i_p}$ denotes the payload carried along the edge $\{i_r, i_p\}$, $\xi_{i_r i_p}$ the observed drone speed, g the gravitational acceleration, ρ the air density, and z and m relate to the drone's rotor blade area and number of rotors, respectively.

Importantly, energy consumption $e_{i_r i_p}$ is subject to uncertainty, as the drone speed $\xi_{i_r i_p}$ is influenced by variable environmental factors, especially wind. To capture this stochasticity, we model energy consumption as a Normal random variable and incorporate chance constraints in our optimization framework. These constraints ensure that, with high probability α (e.g., 90 %), a drone's energy consumption on a trip is not greater than a critical safety threshold, mitigating the risk of mid-flight battery depletion, a vital consideration when balancing routing efficiency, battery lifespan, and operational safety. For notational simplicity, we denote the energy consumption as $e_{i_r i_p}(q_{i_r i_p})$, omitting the explicit dependence on speed here, which will be treated in detail in Section 6.

The problem tackled in this paper, referred to as the Drone Routing Problem with Uncertain Demand and Energy Consumption (DRPUDEC), explicitly builds upon the operational challenges of battery lifespan management and the uncertainty in drone energy use discussed above. The aim is to optimize both ground service and flight operations by ensuring balanced battery utilization while accounting for stochastic customer requests and variable energy consumption.

To handle uncertain demand dynamically, we adopt an MDP framework coupled with Approximate Dynamic Programming (ADP). Additionally, we apply chance constraints to design safe trips that can be completed with high probability under adverse conditions, preventing premature drone returns due to unexpected battery depletion.

Routing plans generated under this framework aim to maximize the number of customers served while minimizing the expected total cost composed of (*a*) the routing costs measured in terms of the total expected distance traveled by drones and (*b*) the lateness cost due to the expected accumulated lateness as the approach allows to serve customer's requests after their soft deadlines. An overview of the notation used throughout the paper is reported in Table 1.

4. Modeling and methodology motivations: a MDP formulation

The problem we face involves two main sources of uncertainty: (1) random customer requests and (2) the variability in drone battery energy consumption. These uncertainties can be modeled using distinct stochastic processes - requests follow a Poisson process, while energy consumption is modeled as Normal random variables. As decisions and uncertainties unfold sequentially over time, the problem inherently calls for a multi-stage stochastic modeling approach that accurately reflects this temporal evolution and the progressive revelation of information.

Using a tailored multi-stage stochastic programming formulation involves generating a scenario tree to represent all possible realizations of uncertainty through stages. In such a tree, the number of non-dominated scenarios can be prohibitive due to the decoupled sources of uncertainty: each scenario built upon the realizations of requests must be replicated as many times as the number of scenarios produced from the realizations of energy consumption along the arcs of the network. Moreover, the energy consumption realizations, in turn, depend on the weight of the parcels transported by the drones. Consequently, the scenario tree can become prohibitively large and computationally intractable.

Monte Carlo methods are commonly used to address stochastic problems involving more than two stages, as they help manage the rapid growth in problem size by sampling representative scenarios rather than enumerating all possibilities. However, even then, these methods can lead to extremely large decision trees (Powell, 2016). This limitation motivates our choice to adopt an MDP framework to capture the dynamics and stochasticity of the problem over a multi-period planning horizon without explicitly enumerating an enormous scenario tree.

MDPs have gained recognition in diverse fields such as economics, telecommunications, engineering, and ecology, branching out from their operations research roots of the 1950s. This broad adoption has driven significant theoretical advancements in the study of MDPs. Also known as stochastic dynamic programming or stochastic control problems, MDPs provide a rigorous framework for sequential decision-making under uncertainty. An MDP model consists of decision epochs, states, actions, transition probabilities, and costs or rewards. Selecting an action in a given state yields a reward and determines the state at the subsequent decision epoch via a transition probability function. Policies or strategies prescribe which action to choose in response to any eventuality at every future decision epoch. In line with this framework, we first provide the mathematical notation for the MDP formulation of the DRPUDEC over a finite, discrete time horizon $\mathcal{T}$:

- $\mathcal{K}$ is the index set of decision epochs, with $k \in \mathcal{K}$;
- $\mathcal{S}$ is the set of system pre-decision states $s_k \in \mathcal{S}$;
- t_k is the time at which a new pre-decision state s_k occurs. These epochs are spaced by a fixed interval Ψ, within which new realizations of requests can be observed; this triggers a new state of pre-decision of the system in $t_k + \Psi$, in which part of the decision made in t_k can to be modified or updated to integrate these newly revealed requests;
- $\mathcal{X}(s_k)$ is the set of feasible actions in state $s_k \in \mathcal{S}$. Each action $x \in \mathcal{X}(s_k)$ determines a post-decision state s_k^x, which represents the new deterministic state to which the system evolves due to the application of x to s_k;
- $P(s_{k+1}|s_k^x, \varsigma_{k+1})$ is the probability transition function from the post-decision state s_k^x to state s_{k+1} given action x and new customer requests ς_{k+1} revealed at decision epoch $k+1$, and modeled as a non-homogeneous Poisson process;
- $c(s_k, x)$ is the immediate cost incurred from taking action x in state s_k and includes travel and customer service penalties.

A pre-decision state $s_k \in \mathcal{S}$ at decision epoch k is composed of:

Table 1
Notation summary.

Problem data	
b, m, n	number of batteries, number of requests, where $r = \{1, \dots, m\}$, and number of drones, respectively
$\mathcal{N}'$	set of customers nodes (locations), i.e., $\mathcal{N}' = \{i_1, i_2, \dots, i_m\}$
$\mathcal{G}$	graph $\mathcal{G} = (\mathcal{N}, \mathcal{A})$, where $\mathcal{N} = \mathcal{N}' \cup \{i_0\}$ is the set of customers and the depot and $\mathcal{A}$ is the set of edges
$d_{i_r i_p}$	distance between locations $i_r, i_p \in \mathcal{N}$
$\mathcal{T}$	time horizon, where $\mathcal{T} = \{0, 1, \dots, T\}$
$\mathcal{B}$	set of homogeneous batteries with $\lvert\mathcal{B}\rvert = b$ and $\beta \in \mathcal{B}$
Δ	set of homogeneous drones, with $\lvert\Delta\rvert = n$ and $\delta \in \Delta$
Ψ	time interval between two consecutive epochs
$\bar{l}$	time range such that customers whose deadline l_r lies within $t_k + \bar{l}$ are considered urgent
ν	time threshold, where $0 \le \nu < \Psi$, used to updated and cancel trips
μ^d, μ^l	monetary cost per unit of distance and lateness, respectively
Battery data	
ρ, e'_β	battery swapping time and recharge ratio, respectively
ω_k^β	number of times, up to epoch k, battery β was swapped
$E_{\min}, E_{\max}$	battery minimum and maximum energy level, respectively
Drone data	
$\mathtt{m}, \mathtt{z}, \mathtt{p}$	number of rotors in a drone, area of the drone's spinning blade, and fluid density of the air, respectively
$v, Q, \bar{\xi}$	drone's weight, capacity, and average speed, respectively
$\xi_{i_r i_p}$	drone's realized speed between locations $i_r \in \mathcal{N}$ and $i_p \in \mathcal{N}$
$e_{i_r i_p}(q_{i_r i_p})$	energy consumption of a drone carrying $q_{i_r i_p}$ and flying between locations $i_r \in \mathcal{N}$ and $i_p \in \mathcal{N}$
M	maximum number of trips that can be assigned to a drone
Trip data	
$c(\gamma), \tau(\gamma), \psi(\gamma)$	total cost, length, and incurred lateness of a trip γ, respectively
$\mathcal{P}_\gamma$	sequence of request nodes visited by trip γ
$\overline{CE}_\gamma$	required energy to perform a trip γ
Request data	
η	delivery service time
i_r, q_r, l_r	request r location, demand, and soft deadline, respectively
State data and variables	
ς_k	vector of new customer request realizations
Ξ	set of all customer request realizations $\varsigma_k \in \Xi$
$V(s_k)$	expected value-to-go (cost or reward-to-go) from state s_k at time t_k
$\tau(s_k, x)$	total traveled distances incurred by action x when system in state s_k at epoch k
$\psi(s_k, x)$	lateness incurred by action x when system in state s_k at epoch k
$\Delta_k, \mathcal{B}_k$	state of each drone and battery, respectively, at epoch k
$\Delta_k^x, \mathcal{B}_k^x$	state of each drone and battery, respectively, at epoch k due to the execution of action x
$\mathcal{D}_k^x$	set of outstanding requests due to the execution of action x at epoch k
$\mathcal{U}_k$	set newly arrived requests at epoch k
$a_\delta(i_r)$	arrival time of drone $\delta \in \Delta$ at location i_r
θ_k^δ	route plan for drone $\delta \in \Delta$ at period k

- the current time $t_k \in \mathcal{T}$;
- the set of outstanding delivery requests $\mathcal{D}_k = \{(i_r, q_r, l_r)\}$;
- the location, battery level, and availability status of each drone;
- the queue of available and recharging batteries at the depot.

An action $x \in \mathcal{X}(s_k)$ specifies:

- the set of drone trips (node index sequences) $\Gamma(s_{k-1}, s_k)$, that have already been planned in s_{k-1} and are due to be dispatched in $t \le t_{k-1} + \Psi + \nu$, where $0 \le \nu < \Psi$, and updated to account for new requests revealed in $(t_{k-1}, t_k]$ and observed in s_k;
- a set of new trips $\gamma \in \Gamma_k$ that are planned in s_k and assigned to available drones;
- a decision regarding whether to recharge or swap the battery upon return.

Actions are constrained by the feasibility of the route under energy constraints modeled through chance constraints related to stochastic energy consumption. Let $V(s_k)$ denote the expected value-to-go (cost or reward-to-go) from state s_k at time t_k. The Bellman equation is:

$$V(s_k) = \min_{x \in \mathcal{X}(s_k)} \left\{ c(s_k, x) + \sum_{\varsigma_{k+1} \in \Xi} P(s_{k+1} | s_k^x, \varsigma_{k+1}) \cdot V(s_{k+1}) \right\}, \tag{1}$$

where $\sum_{\varsigma_{k+1}\in\Xi} P(s_{k+1}|s_k^x,\varsigma_{k+1})\cdot V\left(s_{k+1}\right)$ is the expectation over the set Ξ of all realizations of exogenous information. Eq. (1) can be solved by starting with some terminal condition such as $V_T\left(s_T\right)=0$ for all ending states T, and then stepping backward in time. An iterative algorithm requires at least three nested loops: the first over all states s_k, the second over all actions x, and the third over all future states $sk+1$. Moreover, such an algorithm requires that the one-step probability transition function $P(s_{k+1}|s_k^x,\varsigma_{k+1})$ be known, which involves a summation over the random variables implicit in the one-step transition.

It seems clear that a solution approach that implements this backward procedure is not computationally tractable because of the "curse of dimensionality"(Bellman and Dreyfus, 1959). It is due to the high dimensionality of $\mathcal{S}$, $\mathcal{X}(s_k)$ for each $s_k\in\mathcal{S}$, and ς. The field emerging under names such as ADP, reinforcement learning, neuro-dynamic programming, and heuristic dynamic programming plays a crucial role in partially overcoming this issue. The key ideas in ADP consist of: *a*) designing a rule (policy) for selecting an action x without exploring the set $\mathcal{X}\left(s_k\right)$ of all feasible actions in state s_k, and *b*) determining a new post-decision state s_{k+1} based on observing it from a physical system, given the pair $\left(s_k,x\right)$, or using the probability transition function $P(s_{k+1}|s_k^x,\varsigma_{k+1})$, in which ς_{k+1} is a sample realization of some random variable which is not known when we choose x.

In the following, we describe the main components of the MDP approach: the states (Section 4.1) and the MDP dynamics (Section 4.2). We illustrate the tailored ADP policy designed to solve the problem in Sections 5 and 6.

4.1. States

At each epoch k, the system occupies a pre-decision state s_k, which contains all the relevant information to make and update routing and ground service decisions. Specifically, a state s_k comprises the current time of day t_k, the state of every request, the state of each drone, and the state of each battery at the charging station. The set of requests known at epoch k is partitioned into two subsets: $\mathcal{D}_k$ and $\mathcal{U}_k$. The set $\mathcal{U}_k$ contains the *new requests* that have just appeared at epoch k, while $\mathcal{D}_k$ contains the *outstanding requests*, which are those that arrived before t_k but have not yet been served. A request remains in $\mathcal{D}_k$ until the corresponding trip has started the process of loading the parcel into a drone.

We denote by Δ_k the set of information describing the state of each drone at epoch k. Specifically, with some abuse of notation, for each drone $\delta\in\Delta_k$, we know the assigned route plan θ_k^δ, which consists of a trip schedule $\left\{\gamma_k^\delta(1),\gamma_k^\delta(2),\ldots,\gamma_k^\delta(w)\right\}$, where $w\le M$ is the last trip index assigned to the drone and M is the maximum number of trips that can be assigned to a drone. We point out that θ_k^δ may also include some trips of $\Gamma\left(s_{k-1},s_k\right)$. This enables incorporating some new information revealed during the operations of other flying drones into these non-departed trips. In the following, for the sake of simplicity, we shall omit symbols k and δ and the trip index if they are clear from the context. Throughout the text we use Δ_k to represent the description the state of the drones at epoch k and the set of drones as in $\delta\in\Delta_k$.

During a trip, a drone δ can visit several locations before returning to the depot to perform service tasks. We represent a generic trip γ as a sequence of nodes $\gamma=\{i_0,i_1,\ldots,i_r,\ldots,i_p,i_{p+1}\}$ starting and ending at the depot $i_0=i_{p+1}$. For each node i_r in the trip, we also define the arrival time $a_\delta\left(i_r\right)$. Since the weather conditions influence the flight time in our approach, we shall consider the corresponding expected value of each trip's total time, cost, and lateness. Note that a deadline of $l_r\in\mathcal{T}$ is associated with each delivery. Lateness in service is computed as $\left[a_\delta\left(i_r\right)-l_r\right]^+=\max\left\{0,a_\delta\left(i_r\right)-l_r\right\}$. Once a parcel has been delivered, a drone can execute the remaining deliveries in the trip. We note that the departure time from a node can be defined as the sum of the arrival time and the service time η, which is assumed to be constant for all customers. We can thus associate with each trip a total expected lateness $\psi(\gamma)$. The total expected cost of a trip γ is given by

$$c(\gamma)=\mu^d\cdot\tau(\gamma)+\mu^l\cdot\psi(\gamma), \tag{2}$$

where $\tau(\gamma)$ is the total expected distance traveled by the drone when performing trip γ, and μ^d and μ^l are the coefficients representing the monetary cost per unit of distance and lateness, respectively.

When a drone δ returns to the depot, it is assigned a fully charged battery. We denote by ρ the service time associated with the battery swapping and loading of new parcels. If no fully charged battery is available, i.e., all batteries are being recharged, the drone has to wait for a battery to be available and assigned to it before leaving the depot for another trip. We denote by $\mathcal{B}_k$ the set of information describing the state of each battery at epoch k. Each battery is assigned a parameter ω_k^β, which indicates the number of times the battery has been swapped from the beginning of the time horizon until epoch k. This latter element guarantees a balanced utilization when assigning a battery to a drone if several fully recharged batteries can be assigned.

Formally, we describe a pre-decision state as $s_k=\left(t_k,\mathcal{D}_k,\mathcal{U}_k,\Delta_k,\mathcal{B}_k\right)$. In the initial state s_0, all drones are available at the depot with empty planned trips, batteries are fully charged, and ω_0^β are set to 0 for every battery $\beta\in\mathcal{B}_0$.

4.2. Dynamics of the DRPUDEC

At a decision epoch k, the decision maker observes the pre-decision state s_k and selects an action $x\in\mathcal{X}(s_k)$, where $\mathcal{X}(s_k)$ is the set of feasible actions corresponding to the pre-decision state s_k. An action x consists of two components: (*i*) it updates the routing plan of the drones through the assignment and routing of newly revealed requests $r\in\mathcal{U}_k$ to new trips and to some of the ones already planned in $\Gamma\left(s_{k-1},s_k\right)$, (*ii*) it specifies the battery-drone assignment. If the number of fully recharged batteries is lower than the number of available drones, a partial assignment is made to have some drones wait at the depot. The first action component is subject to the following requirements:

a) the energy level must remain equal to or greater than a safety margin $E_{\min}$ to prevent the drone from returning to the depot prematurely. Since the energy consumption is uncertain, this condition is ensured by the chance constraint paradigm, which requires that the energy consumption to visit all the locations along a given trip is greater than $E_{\min}$, with probability α.

It is important to note that uncertainty in drone energy consumption is not represented by a state variable, which would require a continuous-time framework - where decisions are made at any moment, rather than at discrete intervals - to determine if a drone should return to the depot. To maintain a discrete-time decision process, the decision to abort a trip and make a drone return to the depot is *not* taken by our actions $\mathcal{X}(s_k)$. Instead, the drones autonomously monitor their battery levels at every minute and act accordingly: they return to the depot if the expected battery level after completing the next delivery and returning to the depot will be lower than $E_{\min}$. All customers not visited due to interrupted trips are reconsidered in D_k;

b) the requests served in a trip are selected following two rules: minimizing the distance traveled and reducing the lateness associated with serving requests beyond their stipulated deadline;

c) drones must return to the depot no later than the end of the day, denoted by T.

Executing an action x in a pre-decision state s_k entails the application of a decision rule, resulting in an immediate cost, denoted by $c(s_k, x)$. Here, the cost $c(s_k, x)$ is computed similarly as in Eq. (2). A sequence of decision rules defines a policy, denoted by $\pi = \left(X_0^\pi(s_0), X_1^\pi(s_1), \dots X_T^\pi(s_T)\right)$, and Π denotes the set of all Markovian deterministic policies. The optimal policy is defined as the one that minimizes the total expected cost, given the initial state s_0:

$$\min_{\pi\in\Pi} \mathbb{E}\left[\sum_{k=1}^{T} c(s_k, X_k^\pi(s_k))|s_0\right]. \tag{3}$$

Using the Bellman recursive equation, the optimization problem (3) can be rewritten as:

$$\min_{\pi\in\Pi}\left[\sum_{k=0}^{T-1} \min_{x\in X^\pi(s_k)}\left\{c(s_k, x) + \sum_{\varsigma_{k+1}\in\Xi} P(s_{k+1}|s_k^x, \varsigma_{k+1})\cdot V\left(s_{k+1}\right)\right\}\right]. \tag{4}$$

Algorithm 1 outlines the dynamics of the MDP. Line 1 initializes the initial state s_0, where all drones are available at the depot, all batteries are fully charged, and the first customers are revealed. Line 2 sets the initial epoch k and the total cost c to zero. Then, in line 4, the pre-decision state s_k transitions to post-decision s_k^x by taking action x. This step is detailed in Algorithm 2. Besides the post-decision s_k^x, the procedure of line 4 also returns the drones' routing and the deliveries' lateness costs incurred in the deterministic transition to the post-decision state s_k^x. These values are added to the total cost.

Algorithm 1: Dynamics of the Markov decision process for the DRPUDEC.

input : Set of drones Δ; set of batteries $\mathcal{B}$.
output: Total cost c.

1 $s_0 \leftarrow \left(t_0, \mathcal{D}_0, \mathcal{U}_0, \Delta_0, \mathcal{B}_0\right)$ // setup initial state s_0
2 $k \leftarrow 0,\ c \leftarrow 0$
3 **while** $t_k < T$ **do**
/* deterministic transition to post-decision state s_k^x from s_k */
4 $s_k^x \leftarrow \left(t_k, \mathcal{D}_k^x, \emptyset, \Delta_k^x, \mathcal{B}_k^x\right),\ c(s_k, x) \leftarrow process\_decision\_state(s_k, v)$ // call Algorithm 2
5 $c \leftarrow c + c(s_k, x)$ // update total cost
/* stochastic transition to the pre-decision state s_{k+1} from s_k^x */
6 $\mathcal{U}_{k+1} = \left\{r_{\text{new}} = \left(i_{r_{\text{new}}}, q_{r_{\text{new}}}, l_{r_{\text{new}}}\right)\right\}$ // newly revealed requests
7 $\mathcal{D}_{k+1} \leftarrow \mathcal{D}_k^x$ // update outstanding customers set
8 $\Delta_{k+1} \leftarrow \Delta_k^x$ // update drones' states
9 $\mathcal{B}_{k+1} \leftarrow \mathcal{B}_k^x$ // update batteries' states
10 $t_{k+1} \leftarrow t_k + \Psi$ // update next time period
11 $k \leftarrow k + 1$ // advance to the next epoch
12 **end**
13 **return** c

Following the generation of s_k^x, the procedure entails the implementation of the stochastic transition to the next pre-decision state s_{k+1} at the epoch $k+1$, as shown in lines 6–11, which are detailed in Section 5.2. The steps of the while loop of lines 3–12 are repeated until the end of the time horizon is reached, and the total cost is returned.

5. Solution methodology

Solving problem (4) to optimality is challenging due to the three curses of dimensionality affecting the Bellman equation. Therefore, a tailored ADP policy is required to obtain a solution of the DRPUDEC. There are four fundamental classes of policies (Powell,

2011; Soeffker et al., 2022): lookahead approximation or rollout algorithms (LA or RA), value function approximation (VFA), PFA, and CFA.

LAs or RAs may require significant online computation to assess the future impact of an action. This may entail incorporating information about future realizations of requests in a predictive manner, particularly when a routing decision must be made, as in the DRPUDEC. VFAs are limited by dimensionality, which can become quite large in routing problems with multiple drones. PFAs are best suited when it is possible to design a policy that captures the structure of the problem to provide decisions. Since the DRPUDEC is affected by uncertainties regarding independent random variables (such as requests and energy consumption of drones), defining a policy based on a state-independent rule can limit the ability to select actions that anticipate the future, as it emerges from the results reported in Section 8.2.2.

CFAs modify a deterministic optimization formulation's objective and/or constraints to better account for future uncertainty. This method can be easily applied to the DRPUDEC where multiple optimization models are considered in sequence to build a policy that mitigates the inherent limitations of rolling horizon optimization, namely its tendency towards a rigid and inflexible decision-making process. In fact, in the DRPUDEC, it is crucial to ensure that decisions account for the opportunity to have a sufficient number of drones either at the charging station or flying back to it at crucial times. This way, they can be routed to deliver new requests revealed at each epoch. This aim can be achieved by defining the maximum number of trips that can be assigned to each drone, allowing for more informed decision-making.

In the proposed approach, the feasible regions of the optimization problems presented in Sections 6.4 and 6.5 are manipulated by the specified maximum number of trips. These problems are solved optimally at each epoch, as detailed in the following sections. Moreover, the computational effort required to solve these problems is minimal compared to the duration of an epoch, which allows for real-time practical applications. This motivates us to design a CFA policy for the DRPUDEC. More precisely, in line 4 of Algorithm 2, we specify that drones cannot be assigned more than M trips. This algorithm establishes a CFA policy, denoted as $\pi^{CFA}(M)$, based on the value of M. In general, let $\Pi^{CFA} = \left\{ \pi^{CFA}(M) : M \in \left\{ 1, \dots, \overline{M} \right\} \right\}$ denote the set comprising all CFA policies for $M \in \left\{ 1, \dots, \overline{M} \right\}$, where $\overline{M}$ is set large enough so that a fleet of n drones in the depot can service all outstanding and new requests in each epoch.

Let M^* be the value of $M \in \left\{ 1, \dots, \overline{M} \right\}$ for which the minimum routing and lateness cost is obtained. When $M \neq M^*$, many drones may still be flying at the beginning of the next epoch, which can lead to an accumulation of delivery lateness for new requests. As evidenced by the computational analysis presented in Section 8.1.1, the best value of M is a function of Ψ. We note that the CFA policy class remains consistent despite varying decision timing and changing values of M. Therefore, no modifications to the algorithm procedures are required. Specifically, when Ψ and M change, the CFA policy $\pi^{CFA}(M)$ adapts to these changes, resulting in different computational outcomes.

A myopic policy $\pi^{CFA}(M = +\infty)$ is the one based on an unbounded number of trips that can be assigned and executed by a drone to serve all the outstanding and newly arrived requests in each state s_k. Unlike the case when $M \in \{1, \dots, \overline{M}\}$, where trips starting after $t_k + \Psi + \nu$ are canceled, the myopic policy executes the assigned trip schedule in full, even if some trips begin in subsequent epochs. This myopic policy fails to consider the possibility of a limited number of scheduled trips being executed by each drone.

The CFA mechanism is outlined as follows. In each state s_k, the CFA policy implements a three-stage decomposition decision process:

1. Multi-trip route planning: A set of feasible trips Γ_k is generated by solving deterministic routing problems that respect energy and capacity constraints. In addition, to enable the design of a more anticipatory policy, some trips in $\Gamma(s_{k-1}, s_k)$ assigned to the drones in Δ_{k-1} in time epoch t_{k-1}, whose departing time lies in $[t_k, t_k + \nu]$, where $\nu \geq 0$ is a parameter calibrated through preliminary experiments, are updated with the new information related to requests revealed between $(t_{k-1}, t_k]$. The trip construction considers the trade-off between energy efficiency and service quality.
2. Dynamic trip scheduling and assignment: For each available drone $\delta \in \Delta_k$ in state s_k, we select up to M trips, denoted by $\Gamma_k(\delta)$, that minimize the overall future cost expressed as
$$\hat{V}_k(s_k) = \sum_{\delta \in \Delta_k} \sum_{\gamma \in \Gamma_k(\delta)} \left(\mu^d \cdot \tau(\gamma) + \mu^l \cdot \psi(\gamma) \right).$$
3. Battery assignment problem: At this stage, before a drone starts its designed trip, two battery assignment strategies are executed: 1) the least used fully charged batteries are assigned to the available drones; 2) some batteries are kept unassigned to prevent future battery shortage. These batteries will only be used if the number of urgent customer requests exceeds a certain level.

In the following sections, a detailed description of the deterministic and stochastic transitions of the system is provided.

5.1. Deterministic transition to post-decision state

Algorithm 2 shows the deterministic transition to the post-decision state s_k^x from the pre-decision state s_k, regarding the design of the CFA policy as outlined in Section 5. Line 2 performs Algorithm 5 which inserts some of the requests from $\mathcal{D}_k$ and the newly revealed requests in $\mathcal{U}_k$ into the trips of $\Gamma(s_{k-1}, s_k)$, whose dispatch time is within $[t_k, t_k + \nu]$. The insertion of these requests into existing trips is evaluated based on the minimum insertion distance, ensuring that capacity and energy constraints are satisfied. For each request, the trip that results in the smallest increase in total travel distance and does not cause lateness is selected; otherwise, the request remains unassigned. Algorithm 5 returns updated sets $\mathcal{D}'_k$ and $\mathcal{U}'_k$ if trips in $\Gamma(s_{k-1}, s_k)$ were updated. Line 3 uses the updated sets $\mathcal{D}'_k$ and $\mathcal{U}'_k$ on the trip-building step, which employs chance constraints to account for uncertainty in energy consumption. In this

step, explained in Section 6.1, a backward heuristic is used to build trips, greedily adding nodes by non-decreasing distance or time to deadline. The backward heuristic generates the set Γ_k of trips that are not dominated, which is used as input for the procedure of line 4 that selects up to M trips per drone from this set.

In the procedure *select_trips*, we initially solve a set packing-based problem that seeks to identify the set $\Gamma'_k \subseteq \Gamma_k \cup \Gamma'(s_{k-1}, s_k)$ that serves the largest number of requests, with the highest priority given to those that are urgent, defined as those for which the soft deadline is near to the current epoch time. The solution of this model consists of at most M trips assigned to a drone. A second model is then solved, this time using the set $\Gamma_k \cup \Gamma'(s_{k-1}, s_k)$ specifying which customers must be served. This model aims to select at most M trips per drone that avoid overlapping customer visits while minimizing the total routing cost. Further details regarding the algorithm in line 4 are provided in Section 6.3.

Algorithm 2: Process decision state (*process_decision_state*).

input : Pre-decision state $s_k = (t_k, \mathcal{D}_k, \mathcal{U}_k, \Delta_k, \mathcal{B}_k)$; trip starting time threshold parameter ν.
output: Post-decision state $s_k^x = (t_k^x, \mathcal{D}_k^x, \emptyset, \Delta_k^x, \mathcal{B}_k^x)$; cost $c(s_k, x)$.

1 $\tau(s_k, x) \leftarrow 0,\ \psi(s_k, x) \leftarrow 0$
2 $(\mathcal{D}'_k, \mathcal{U}'_k) \leftarrow update\_trips(\mathcal{D}_k \cup \mathcal{U}_k, \nu)$ // update sets by Algorithm 5
3 $\Gamma_k \leftarrow build\_trips(\mathcal{D}'_k \cup \mathcal{U}'_k)$ // build set Γ_k by Algorithm 3
4 $\Gamma^*_k \leftarrow select\_trips(\mathcal{D}'_k \cup \mathcal{U}'_k, \Gamma_k, |\Delta|, M)$ // find set Γ^*_k by Algorithm 6
5 $\{\theta^\delta_k : \delta \in \Delta_k\} \leftarrow assign\_trips(\Gamma^*_k, \Delta_k)$ // assign trips to drones by Algorithm 7
6 **for each** $\delta \in \Delta_k$ **do**
  /* execute state machine, depicted by Fig. 1, of drone δ for Ψ units of time. During this execution, update sets $\Delta_k^x, \mathcal{B}_k^x, \mathcal{D}_k^x$ and update costs $\tau(s_k, x)$ and $\psi(s_k, x)$ accordingly. Also, update ω_k^β for every battery β swapped */
7 $execute\_drone\_state\_machine(\delta, \theta^\delta_k, \mathcal{B}_k^x, \tau(s_k, x), \psi(s_k, x))$
  /* after running drone δ's state machine for Ψ units, trips without lateness and starting in $[t_k, t_k + \nu]$ are carried over to the next state. The remaining ones are canceled, and their customers are returned to the set $\mathcal{D}_k^x$ */
8 $\mathcal{D}_k^x \leftarrow \mathcal{D}_k^x \cup cancel\_scheduled\_trips(\delta)$
9 **end**
10 **return** $s_k^x, \mu^d \cdot \tau(s_k, x) + \mu^l \cdot \psi(s_k, x)$

Solving the previous model allows us to find the set Γ^*_k of least routing cost drone trips, whereby the maximum number of customers are visited. This set can be partitioned into n trip subsets, each representing a potential assignment of M trips to a drone. When a trip schedule is assigned to a drone, additional lateness in serving customers on a trip can be accumulated due to the sub-optimal order in which the trips are executed in each schedule. Therefore, an optimal permutation of the trips assigned to a drone must be identified to minimize customer service lateness. This is achieved by exhaustively considering all possible trip orders and selecting the schedule with the least lateness. The least lateness trip schedules are then used in a third model, which is tasked with assigning each schedule to a drone in a manner that minimizes overall lateness while accounting for the state of the drones at the current epoch time. These steps are executed on line 5, where θk^δ is the trip schedule assigned to drone $\delta \in \Delta_k$. We observe that scenarios with fewer trip schedules than drones are possible. Therefore, assignment solutions where one or more drones are not used are also valid. Further details of the algorithm referenced in line 5 are provided in Section 6.6.

After the assignment step, each drone $\delta \in \Delta_k$ executes its assigned route plan θ^δ_k, if any, following the algorithm outlined in line 7. Fig. 1 depicts the dynamic of the DRPUDEC, describing a trip schedule that could be executed by a drone in any potential scenario. This is referred to as the drone finite-state machine. It operates for each $\delta \in \Delta_k$ over a duration of Ψ units of time, during which it updates sets $\Delta_k^x, \mathcal{B}_k^x, \mathcal{D}_k^x$. Additionally, it aggregates the cost $c(s_k, x)$ incurred by the trips executed by drone δ within the Ψ time span. Upon completing its operation after Ψ units of time, the drone idles at its current state and position within the state machine and resumes from the same state and position in the subsequent epoch $k + 1$.

Following the execution of the drone δ state machine for a duration of Ψ, any future trips dispatched in $[t_{k+1}, t_{k+1} + \nu]$ that will not be late are added to a list to be carried over into the next state. At the same time, any trips that are late or start after $t_{k+1} + \nu$ are canceled, as outlined in line 8 of Algorithm 2. We observe that trips currently underway by a drone and the ones starting before $t_k + \Psi$ are not affected. Then, all customers associated with the unexecuted trips are returned to the set $\mathcal{D}_k^x$. After executing all drones for Ψ units of time, the post-decision state s_k^x and its corresponding cost and lateness are returned.

In the context of the drone finite-state machine, starting a trip, fulfilling requests in new or existing trips, and replacing batteries are triggered by the respective action condition, resulting in a transition of the state. For instance, at the beginning of Algorithm 1, once the initial decision state s_0 is initialized, all drones $\delta \in \Delta$ are on the state "Standby". In this state, a drone δ waits for a trip to be assigned to it, which is done in line 5 of Algorithm 2. Then, once the drone finite-state machine is executed in line 7, the drone gets the first trip of the assigned schedule, and the state transitions to "Ready to fly".

During flight, the drone remains in this state until reaching a vertex, either a depot or a customer location. If the vertex is a customer, then the state changes to "Delivering", and stays on it for the duration of the service time η. In the case of the absence of

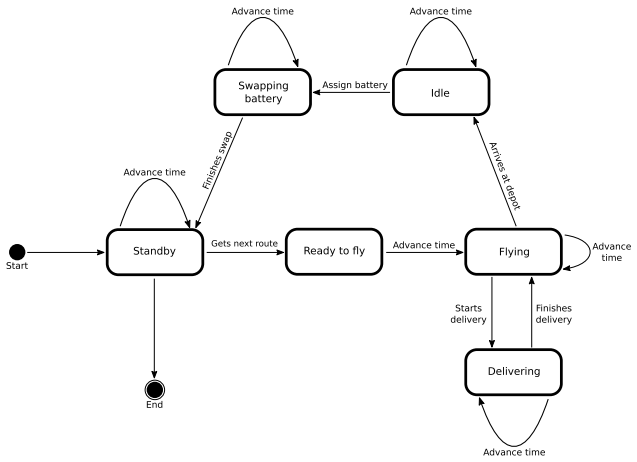

Fig. 1. Drone execution finite-state machine.

the customer, we assume that the delivery can be made, for example, in front of their place. After finishing the delivery, the state transitions back to "Flying", and continues until it reaches the next vertex. If the vertex is the depot, then the drone has finished a trip, and its state transitions to "Idle", where it waits for its depleted battery to be swapped with a fully charged one. Once a new battery is assigned to the idle drone, its state transitions to "Swapping battery", which is done for the duration ρ. After the battery swapping, the drone state returns to "Standby" wherein it awaits the loading of all scheduled parcels before continuing with the next scheduled trip and repeats the transitions again. If no trip is left, then the drone waits for the next trip assignment of the next epoch. The empty state "End" is only reached at the end of the MDP execution.

5.2. Stochastic information and transition to pre-decision state

After the execution of an action x triggering the deterministic transition to the post-decision state s_k^x, the arrival of random exogenous information represented by set $\mathcal{U}_{k+1}$ determines the stochastic transition from s_k^x to s_{k+1}. The time associated with the new state is defined as $t_{k+1} = t_k + \Psi$, with Ψ denoting a given time step eventually calibrated based on the process describing the arrival of new requests during the day. Our choice differs from other contributions that associate the new decision point with the arrival of a new request. The motivation comes from the difference in the considered problems. During some hours of the day, the frequency of arrival of requests can be very high, making the continuous update of the routing plans useless.

The transition to the next decision state s_{k+1} starting from s_k^x entails updating the sets of requests, which is done in lines 6–11 of Algorithm 1. The set of new requests appearing between t_k and t_{k+1}, denoted by set $\mathcal{U}_{k+1}$, is updated in line 6 with the newly-arriving random requests. Line 7 updates the set of outstanding requests $\mathcal{D}_{k+1}$ with the requests from epoch k, i.e., $\mathcal{D}_{k+1} \leftarrow \mathcal{D}_k^x$. Finally, the set of drones Δ_{k+1} is updated to include the new states of drones for the next period encompassing their flying operations and/or recharge operations at the depot, and $\mathcal{B}_{k+1}$ is also updated to consider the new state of the batteries.

6. Algorithm components

This section provides a comprehensive, step-by-step description of the procedures invoked in Algorithm 2 to perform the action derived from the adopted CFA policy. As previously indicated, it is composed of five steps: a trip generation procedure, a trip update procedure, an algorithm to select trips maximizing the number of customers served, a procedure to assign trips to drones in order to visit all customers previously selected while minimizing total costs (distance and delay), and finally a battery assignment to the drones that are scheduled to fly.

6.1. Trip construction heuristic

This section details the procedure *build_trips* called in line 3 of Algorithm 2. Let $\mathcal{G}_k = (\mathcal{N}_k, \mathcal{A}_k)$ be the undirected subgraph of G induced by the delivery locations known at the decision epoch t_k, besides the depot i_0, where $\mathcal{N}_k \subseteq \mathcal{N}$, $\mathcal{N}'_k = \mathcal{N}_k \setminus \{i_0\}$, and $\mathcal{A}_k \subseteq \mathcal{A}$. Formally, the node set is defined as $\mathcal{N}'_k = \{i_r \mid r = (i_r, q_r, l_r) \in \mathcal{D}_k \cup \mathcal{U}_k\}$ and the set of edges is defined as $\mathcal{A}_k = \{\{i_r, i_p\} \mid i_r, i_p \in \mathcal{N}_k\}$. To build trips, we propose a label-setting algorithm. This algorithm relies on a backward approach and considers energy saving. Let us consider a generic trip $\gamma = \{i_0, i_1, \ldots, i_r, \ldots, i_p, i_{p+1}\}$ visiting the sequence of customer nodes $\mathcal{P}_\gamma = \{i_1, \ldots, i_p\}$, where $i_0 = i_{p+1}$. We define the trip feasible if: *i*) the total payload is lower than the drone's capacity and *ii*) the state of charge of the battery at the end of the trip is greater than or equal to a given threshold E_{min}. Given that energy consumption is random, we employ the paradigm of chance constraints (CC) to build safe trips. In particular, we impose that:

$$\mathbb{P}(\widetilde{CE}_\gamma \geq E_{min}) \geq \alpha,$$

where $\widetilde{CE}_\gamma$ denotes the total energy consumption random variable, and α is a probability value in (0, 1) used to calibrate the risk attitude. In particular, the higher this value, the more risk-averse the decision maker is and the more conservative the corresponding solutions. We deal with the CC assuming that the random variables in calculating $\widetilde{CE}_\gamma$ follow a Normal distribution and are independent. Under this assumption, the CC admits a deterministic equivalent reformulation that can be easily derived (e.g., Prékopa, 1970):

$$\mathbb{E}[\widetilde{CE}_\gamma] \geq E_{min} + \Phi^{-1}(\alpha) SDV(\widetilde{CE}_\gamma),$$

where $\Phi^{-1}(\alpha)$ is the α-quantile of the Normal standard distribution function and SDV the standard deviation. In the formula, the expected total energy consumption and its standard deviation can be derived by exploiting the properties of the Normal random variables. Specifically,

$$\mathbb{E}\left[\widetilde{CE}_\gamma\right] = \mathbb{E}\left[\sum_{h=0}^{|\mathcal{P}_\gamma|-1} \tilde{e}_{i_h,i_{h+1}}(q_{i_h,i_{h+1}}) + \tilde{e}_{i_{|\mathcal{P}_\gamma|},i_0}\right] = \sum_{h=0}^{|\mathcal{P}_\gamma|-1} \bar{e}_{i_h,i_{h+1}}(q_{i_h,i_{h+1}}) + \bar{e}_{i_{|\mathcal{P}_\gamma|},i_0} v. \tag{5}$$

Here, $|\mathcal{P}_\gamma|$ denotes the number of served requests along the trip, whereas $\bar{e}_{i_h,i_{h+1}}(q_{i_h,i_{h+1}})$ is the expected energy consumption, that depends on the total payload $q_{i_h,i_{h+1}}$ on edge $\{i_h, i_{h+1}\}$ (including the drone weight). The last term in the sum accounts for the energy the drone consumes to fly back to the depot starting from the last location, considering only the drone weight v.

The load on a given edge $\{i_h, i_{h+1}\}$ also considers the payloads of the deliveries associated with nodes visited after i_h. In our case, we assume that the energy consumption is a linear function of the payload. Moreover, the SDV of the estimated cost $\widetilde{CE}_\gamma$ is computed under the independence assumption between edge costs. It accounts for the variances along each path edge, weighted by the squared cumulative payloads.

The information introduced above is used in the trip construction phase. The trips are built by implementing a dynamic programming backward label setting algorithm with different strategies for node extensions by distance and urgency. In this heuristic, trips are constructed in a backward manner because we can compute a trip's exact payload and energy consumption while it is being constructed without the need for additional computational burden. In particular, for each node i_r explored during the trip construction we introduce two sets, $\mathcal{I}^d(i_r)$ and $\mathcal{I}^l(i_r)$. In the former, nodes i_p connected to i_r are sorted according to non-decreasing values of the distance from i_r to i_p, whereas in the latter, according to non-decreasing $l_p - t_k$ values. In the first case, we are interested in minimizing distances, while in the second one, the aim is to prioritize the most urgent requests to minimize lateness.

The backward labeling heuristic, presented by Algorithm 3, aims to create trips fulfilling the chance constraint $\mathbb{P}(\widetilde{CE}_\gamma \geq E_{min}) \geq \alpha$ and generate trips that are optimized for the distance traveled or total lateness. Algorithm 3 finds a set of non-dominated trips, denoted as Γ_k. The domination rule states that if two distinct trips $\gamma_1 = \left\{i_0, i_1^1, \ldots, i_p^1, i_{p+1}\right\}$ and $\gamma_2 = \left\{i_0, i_1^2, \ldots, i_p^2, i_{p+1}\right\}$ are given, the latter is dominated by the former if the following conditions hold:

a) $\mathcal{P}_{\gamma_1} = \mathcal{P}_{\gamma_2}$;
b) $\sum_{h=0}^{|\mathcal{P}_{\gamma_1}|-1} \bar{e}_{i_h^1 i_{h+1}^1}\left(\sum_{l=h+1}^{|\mathcal{P}_{\gamma_1}|} q_{i_l^1}\right) \leq \sum_{h=0}^{|\mathcal{P}_{\gamma_2}|-1} \bar{e}_{i_h^2 i_{h+1}^2}\left(\sum_{l=h+1}^{|\mathcal{P}_{\gamma_2}|} q_{i_l^2}\right)$, i.e., the total average energy consumption of trip γ_1 is not greater than that of trip γ_2.

To build set Γ'_k, Algorithm 3 begins with a set Γ'_k, which is composed of trips that can be extended, i.e., trips where a customer may still be inserted. At the beginning of the algorithm, set Γ'_k starts with an empty trip $\gamma = \{i_0, i_{p+1}\}$. At each iteration of Algorithm 3, up to Σ nodes adjacent to the last backward-inserted node i_r are selected to be inserted in the current trip γ, creating up to Σ new trips. The vertices selected to be inserted in the current trip are denoted by $\mathcal{V}$. This set is found by Algorithm 4, which is called in line 7 of Algorithm 3. If set $\mathcal{V}$ is empty, indicating that the procedure found no node in line 7, and thus that trip γ can no longer be backward extended, then γ is added to Γ_k, provided that γ is not dominated by any other trip in Γ_k. If set $\mathcal{V}$ is not empty, a new trip γ' is generated for each $i \in \mathcal{V}$ by inserting i backward into γ. Thereafter, all non-dominated trips created by this step are inserted into Γ'_k. These steps are reiterated until Γ'_k becomes empty, indicating that no trip can be further extended. If there are still vertices that are not visited by any trip in Γ_k, then the algorithm is executed again but only considering unvisited vertices.

The selection process for vertices to be inserted into a trip γ by the backward heuristic is illustrated in Algorithm 4. This method finds a set of $\mathcal{V}$ containing up to Σ vertices to be inserted in γ. Let i_r be the last backward-inserted vertex in the trip γ, and $i_p \in \mathcal{I}^d(i_r)$ (or $i_p \in \mathcal{I}^l(i_r)$). Vertex i_p is only added to $\mathcal{V}$ if its insertion in γ does not violate the drone's capacity and the maximum energy consumption, as defined by inequalities (6) and (7). We observe that inequality (7) also accounts for the energy needed to return to the depot. If all vertices in the neighbor set of i_r (either $\mathcal{I}^d(i_r)$ or $\mathcal{I}^l(i_r)$) are explored or $|\mathcal{V}| = \Sigma$, then Algorithm 4 stops and returns the set $\mathcal{V}$ as it is, even if it is empty.

$$\sum_{l=1}^{|\mathcal{P}_\gamma|} q_{i_l} + q_{i_p} \leq Q, \tag{6}$$

$$\bar{e}_{i_0 i_p}\left(q_p + \sum_{l=|\mathcal{P}_\gamma|}^{1} q_{i_l}\right) + \bar{e}_{i_p i_{|\mathcal{P}_\gamma|}}\left(\sum_{l=|\mathcal{P}_\gamma|}^{1} q_{i_l}\right) + \sum_{h=|\mathcal{P}_\gamma|}^{2} \bar{e}_{i_h i_{h-1}}\left(\sum_{l=h-1}^{1} q_{i_l}\right) + \bar{e}_{i_1 i_{n+1}} v \leq E_{\max} - \Phi^{-1}(\alpha)\sqrt{\sigma^2_{i_0 i_p} + \sigma^2_{i_p i_{|\mathcal{P}_\gamma|}} + \sum_{h=|\mathcal{P}_\gamma|}^{2} \sigma^2_{i_h i_{h-1}} + \sigma^2_{i_1 i_{n+1}}}. \tag{7}$$

In inequality (7), $\sigma^2_{i_h i_{h'}}$ is the variance of the energy consumption over an edge $\{i_h, i_{h'}\} \in \mathcal{A}$.

Algorithm 3: Backward heuristic (*build_trips*).

input : Set of outstanding and newly arrived requests $\mathcal{D}_k \cup \mathcal{U}_k$ at epoch k.
output: Set of trips Γ_k.

Define the maximum size of set $\mathcal{V}$, represented by parameter Σ
$\Gamma_k \leftarrow \emptyset$
$\Gamma'_k \leftarrow \{i_0, i_{p+1}\}$ // set of trips being build. Initially, set Γ'_k has only trip $\{i_0, i_{p+1}\}$
while $\Gamma'_k \neq \emptyset$ **do**
  $\gamma \leftarrow select\_trip(\Gamma'_k)$ // select a trip from set Γ'_k
  $\Gamma'_k \leftarrow \Gamma'_k \setminus \gamma$ // remove γ from Γ'_k
  $\mathcal{V} \leftarrow select\_vertices\_to\_insert(\gamma, \Sigma)$ // select vertices by Algorithm 4
  if $\mathcal{V} = \emptyset$ **and** γ *is not dominated by any trip in* Γ_k **then**
    $\Gamma_k \leftarrow \Gamma_k \cup \gamma$ // add γ to the set of trips to be returned
  else if $\mathcal{V} \neq \emptyset$ **then**
    for each $i \in \mathcal{V}$ **do** // create $|\mathcal{V}|$ new trips by inserting each i in γ
      $\gamma' \leftarrow \gamma \cup \{i\}$ // i is inserted after the last backward-inserted vertex i_r
      $\Gamma'_k \leftarrow \Gamma'_k \cup \gamma'$ // add γ to the set of trips being build
    end
  end
end
return Γ_k

Algorithm 4: *select_vertices_to_insert*.

input : Trip γ; Σ, the maximum size of set $\mathcal{V}$.
output: Set of vertices $\mathcal{V}$.

Let $i_r \in \gamma$ be the first location to be visited by the trip γ, after the depot i_0
$\mathcal{V} \leftarrow \emptyset$
while $|\mathcal{V}| < \Sigma$ **and** *there is a neighbor vertex of* i_r *to be explored* **do**
  /* select i_p from the correct neighbor set, following the sorting policy */
  $i_p \leftarrow select\_first\_neighbor(\mathcal{I}^d(i_r))$ **or** $i_p \leftarrow select\_first\_neighbor(\mathcal{I}^l(i_r))$
  if *inequalities (6) and (7) hold when inserting* i_p *into* γ **then**
    $\mathcal{V} \leftarrow \mathcal{V} \cup i_p$
  end
  $\mathcal{I}^d(i_r) \leftarrow \mathcal{I}^d(i_r) \setminus i_p$ **or** $\mathcal{I}^l(i_r) \leftarrow \mathcal{I}^l(i_r) \setminus i_p$ // remove i_p from the correct neighbor set
end
return $\mathcal{V}$

6.2. Trip update heuristic

This section describes the procedure *update_trips*, outlined in Algorithm 5 and called in line 2 of Algorithm 2. Let $\mathcal{D}_k$ denote the unassigned outstanding requests in s_k and let $\mathcal{U}_k$ represent the newly revealed requests in s_k. For each $r \in \mathcal{D}_k \cup \mathcal{U}_k$ and for each $\gamma \in \Gamma(s_{k-1}, s_k)$, if the insertion is feasible, let $\Delta\tau(\gamma, i_r)$ be the increase in the distance of trip γ resulting from the insertion of node i_r (associated with request r) between nodes $i_h \in \gamma$ and $i_{h+1} \in \gamma$ using the minimum insertion distance criterion. Similarly, let $\Delta\psi(\gamma, i_r)$ denote the increase in the lateness of γ due to the insertion of i_r.

Line 4 of Algorithm 5 identifies a trip $\gamma \in \Gamma(s_{k-1}, s_k)$ (if any) and the corresponding position where i_r can be feasibly inserted with minimum $\Delta\tau(\gamma, i_r)$. In line 5, it is verified whether γ can accommodate r without lateness. If so, i_r is inserted into γ. Finally, the sets $\mathcal{D}'_k \cup \mathcal{U}'_k$ are updated accordingly.

6.3. Trip selection problem

This section provides a detailed description of the procedure *select_trips* invoked in line 4 of Algorithm 2. As the procedure presented in Section 6.1 finds several trips, selecting and assigning the most promising ones to the drones is necessary. Algorithm 6 depicts the procedure for selecting these trips. First, as we want to serve as many (urgent) requests as possible, we solve a Set Packing-Based Problem (SPBP), which aims to maximize the number of (urgent) requests selected. Second, we use the requests served in the trips of an optimal solution of the SPBP as input for a non-overlapping Δ-Trip Set Selection Problem (DTSSP). The DTSSP aims to minimize non-overlapping trip routing costs while ensuring that urgent requests selected in the optimal solution of the SPBP are covered by at least one trip.

Algorithm 5: Trip update Heuristic (*update_trips*).

input : Set of outstanding and newly arrived requests $\mathcal{D}_k \cup \mathcal{U}_k$ at epoch k; Threshold ν.
output: Updated set of outstanding and newly arrived requests $\mathcal{D}'_k \cup \mathcal{U}'_k$ at epoch k.

1 $\mathcal{D}'_k \leftarrow \mathcal{D}_k$, $\mathcal{U}'_k \leftarrow \mathcal{U}_k$
2 $\Gamma'(s_{k-1}, s_k) \leftarrow \{\gamma \in \Gamma(s_{k-1}, s_k) : \text{no lateness and dispatching time} \in [t_k, t_k + \nu]\}$
3 **for each** $r \in \mathcal{D}_k \cup \mathcal{U}_k$ **do**
4 $\gamma \leftarrow find\_best\_trip\_for\_request(\Gamma'(s_{k-1}, s_k), i_r)$
5 **if** $\gamma \neq \emptyset$ *and* $\Delta\psi(\gamma, i_r) = 0$ **then**
6 $\gamma \leftarrow \gamma \cup \{i_r\}$ `// i_r is inserted on γ in the correct position found at line 4`
7 $\mathcal{D}'_k \cup \mathcal{U}'_k \leftarrow (\mathcal{D}'_k \cup \mathcal{U}'_k) \backslash \{r\}$
8 **end**
9 **end**
10 **return** $\mathcal{D}'_k \cup \mathcal{U}'_k$

In Algorithm 6, the selected trips are not yet assigned to drones; rather, they are selected as candidates for an assignment because they are built by ignoring which drones are flying and which ones are idle. Instead, Algorithm 6 considers that all drones are available at the depot to make the models simpler and easier to solve. The method used to assign the chosen trips to the drones is detailed in Section 6.6. The mathematical formulations of the SPBP and DTSSP are shown in Sections 6.4 and 6.5, respectively.

Algorithm 6: Select trips (*select_trips*).

input : Set of outstanding and newly arrived requests $\mathcal{D}_k \cup \mathcal{U}_k$; Set of trips Γ_k; number of drones n; maximum number of trips per drone M.
output: Set of trips Γ^*_k without repeated requests.

1 $\mathcal{D}^{pr}_k \cup \mathcal{U}^{pr}_k, \mathcal{D}^{npr}_k \cup \mathcal{U}^{npr}_k \leftarrow split(\mathcal{D}_k \cup \mathcal{U}_k)$ `// split requests into priority` $\mathcal{D}^{pr}_k \cup \mathcal{U}^{pr}_k$ `and non-priority` $\mathcal{D}^{npr}_k \cup \mathcal{U}^{npr}_k$
2 $\mathcal{D}'_k \cup \mathcal{U}'_k \leftarrow$ SPBP$(\mathcal{D}^{pr}_k \cup \mathcal{U}^{pr}_k, \mathcal{D}^{npr}_k \cup \mathcal{U}^{npr}_k, \Gamma_k, n, M)$ `// solve model (8a)–(8h) and find set` $\mathcal{D}'_k \cup \mathcal{U}'_k$
3 $\Gamma^*_k \leftarrow$ DTSSP$(\mathcal{D}_k \cup \mathcal{U}_k, \mathcal{D}'_k \cup \mathcal{U}'_k, \Gamma_k, n, M)$ `// solve model (9a)–(9b) and find set of trips` Γ^*_k
4 **return** Γ^*_k

6.4. Set packing-based problem

Let $\mathcal{D}^{pr}_k \cup \mathcal{U}^{pr}_k = \{r \mid r \in \mathcal{D}_k \cup \mathcal{U}_k,\ l_r \in [0, \min\{t_k + \bar{l}, T\}]\}$ be the set of priority outstanding and newly arrived requests at epoch k, i.e., requests whose soft deadline lies within $\left[0, \min\{t_k + \bar{l}, T\}\right]$, where $\bar{l}$ is a non-negative value in minutes. It thus follows that the set $\mathcal{D}^{npr}_k \cup \mathcal{U}^{npr}_k = \{r \mid r \in \mathcal{D}_k \cup \mathcal{U}_k \wedge r \notin \mathcal{D}^{pr}_k \cup \mathcal{U}^{pr}_k\}$ is defined as the set of non-priority requests, that is, requests for which the delivery can be postponed to after $t_k + \bar{l}$. Moreover, consider $\Gamma_{k,r} \subseteq \Gamma_k$ as the set of trips containing request $r \in \mathcal{D}_k \cup \mathcal{U}_k$. The SPBP can be formulated with binary decision variables $y_{\gamma j}$ equal to one if trip $\gamma \in \Gamma_k$ is assigned to drone j, and z_r equal to one if customer $r \in \mathcal{D}_k \cup \mathcal{U}_k$ is visited by at least one trip. The problem can then be modeled as:

$$\text{(SPBP)} \quad \max \sum_{r \in \mathcal{D}^{pr}_k \cup \mathcal{U}^{pr}_k} \mu^{pr} z_r + \sum_{r \in \mathcal{D}^{npr}_k \cup \mathcal{U}^{npr}_k} \mu^{npr} z_r \tag{8a}$$

subject to

$$\sum_{j=1}^{n} y_{\gamma j} \leq 1, \qquad \forall \gamma \in \Gamma_k \tag{8b}$$

$$\sum_{\gamma \in \Gamma_k} y_{\gamma j} \leq M, \qquad j = 1, \ldots, n \tag{8c}$$

$$z_r \leq \sum_{\gamma \in \Gamma_{k,r}} \sum_{j=1}^{n} y_{\gamma j}, \qquad \forall r \in \mathcal{D}_k \cup \mathcal{U}_k \tag{8d}$$

$$\sum_{\gamma \in \Gamma_k} \left(\sum_{h=0}^{|\mathcal{P}_\gamma|-1} \bar{t}_{i_h i_{h+1}} \right) y_{\gamma j} \leq \sum_{\gamma \in \Gamma_k} \left(\sum_{h=0}^{|\mathcal{P}_\gamma|-1} \bar{t}_{i_h i_{h+1}} \right) y_{\gamma j+1}, \qquad j = 1, \ldots, n-1 \tag{8e}$$

$$t_k + \sum_{\gamma \in \Gamma_k} \left(\sum_{h=0}^{|\mathcal{P}_\gamma|-1} \bar{t}_{i_h i_{h+1}} \right) y_{\gamma j} + \rho \left(\sum_{\gamma \in \Gamma_k} y_{\gamma j} - 1 \right) \leq T, \qquad j = 1, \ldots, n \tag{8f}$$

$$y_{\gamma j} \in \{0,1\}, \qquad \forall \gamma \in \Gamma_k,\ j = 1,\dots,n \tag{8g}$$

$$z_r \in \{0,1\}, \qquad \forall r \in \mathcal{D}_k \cup \mathcal{U}_k. \tag{8h}$$

The objective function (8a) seeks to maximize the weighted number of customers visited on selected trips. In this context, the coefficients μ^{pr} and μ^{npr} represent the weights given to urgent and non-urgent customers. Each trip must be assigned to at most a single drone, following constraints (8b). The number of trips assigned to a drone is limited to a maximum of M, as guaranteed by constraints (8c). Constraints (8d) link variables z_r to $y_{\gamma j}$ and ensure that z_r is equal to one only if customer r is contained in at least one selected trip. Constraints (8e) break solution symmetry concerning the drone indices $j = 1,\dots,n$. Introducing these constraints ensures that the sets of trips are assigned to drones in an order that reflects the increasing duration of the trips. Constraints (8f) ensure that the total time required for all trips assigned to a drone does not exceed the time limit. We note that constraints (8f) do not consider the time for scheduling the sequence of trips assigned to the same drone. This aspect is addressed in Section 6.6. Finally, constraints (8g) and (8h) define the domain of the variables.

6.5. Non-overlapping Δ-trip set selection problem

This problem aims to construct as many sets of trips as the drones available at the depot while ensuring that no customer is visited more than once on a trip. To achieve this, we optimize the following mathematical formulation and verify that the feasibility condition is satisfied. Specifically, the SPBP maximizes the number of (urgent) requests to be served by selecting trips that may overlap. The selected requests are now fixed and used to solve another model to minimize the routing costs. Let $\Gamma'_k \subseteq \Gamma_k$ denote the set of trips in an optimal solution to the model (8a)–(8h). Furthermore, let $\mathcal{D}'_k \cup \mathcal{U}'_k \subseteq \mathcal{D}_k \cup \mathcal{U}_k$ be the set of requests contained in at least one trip of Γ'_k. Then, the DTSSP can be formulated as:

$$\text{(DTSSP)}\ \min \sum_{\gamma \in \Gamma_k} c(\gamma) \sum_{j=1}^{n} y_{\gamma j} \tag{9a}$$

subject to (8b), (8c), (8e), (8f), (8g), and to

$$\sum_{\gamma \in \Gamma_{k,r}} \sum_{j=1}^{n} y_{\gamma j} \geq 1, \qquad \forall r \in \mathcal{D}_k \cup \mathcal{U}'_k \tag{9b}$$

$$\sum_{\gamma \in \Gamma_k} y_{\gamma j} \leq \sum_{\gamma \in \Gamma_k} y_{\gamma j+1} + 1, \qquad j = 1,\dots,n-1 \tag{9c}$$

$$\sum_{\gamma \in \Gamma_k} y_{\gamma n} \leq \sum_{\gamma \in \Gamma_k} y_{\gamma 1} + 1. \tag{9d}$$

Objective function (9a) minimizes the total routing costs of the selected trips. The covering constraints (9b) ensure that requests in $\mathcal{D}'_k \cup \mathcal{U}'_k$ must be contained in at least one selected trip. Constraints (9d) aim to achieve a balance in utilizing drones. This is achieved by imposing that a drone can only be assigned p trips, with $p > 1$, if at least $p-1$ trips were assigned to drone $j+1$. It is essential to impose constraints (9d) to guarantee that the last drone, identified by index n, also complies with the balanced utilization of drones prescribed by constraints (9d). To accelerate the resolution of the mathematical formulation (9a)–(9d), the solution of the SPBP model is employed as a warm start. If the optimal solution of (9a)–(9d) contains trips that visit the same customer more than once, this visit is removed from the trip with the fewest customers visited. If the trip becomes empty as a result, it is eliminated.

6.6. Trip assignment problem

Let $\Gamma^*_k \subseteq \Gamma_k$ be the set of trips, without repeated requests, selected by solving the mathematical formulation (9a)–(9b), such that $\bigcup_{j=1}^{n} \Gamma^*_{k,j} = \Gamma^*_k$, where $\Gamma^*_{k,j}$ is the subset of trips that can be assigned to drone $j = 1,\dots,n$. We observe that $\Gamma^*_{k,j_1} \cap \Gamma^*_{k,j_2} = \emptyset$ for $1 \leq j_1, j_2 \leq n$ and $j_1 \neq j_2$. Each subset $\Gamma^*_{k,j}$ does not consider the current status of drone j. Consequently, it could be a suboptimal decision to assign it to j. Instead, better assignments are identified using the procedure described in Algorithm 7, taking into account the least lateness of each trip schedule obtained with a permutation of the trips, and the state of the drones at the current time of the epoch. The trips assigned by Algorithm 7 will be executed by the drone upon its return to the depot, where it will also swap its battery.

We denote by $\bar{\Pi}(\Gamma^*_{k,j}, t_k)$ the permutation operator that aims to find the best permutation of the trips in each subset $\Gamma^*_{k,j}$, with $j = 1,\dots,n$, to achieve the minimum lateness in the fulfillment customer requests. The permutation operator $\bar{\Pi}(\Gamma^*_{k,j}, t_k)$ enumerates all possible trip schedules assigned to drone $j = 1,\dots,n$ and selects the one that minimizes the total lateness in servicing all requests across all trips, starting from the current time t_k. If no lateness occurs in any permutation, then the routes are sorted in nonincreasing order of the number of urgent customers. In case of a tie, the route with the earliest deadline is given priority.

Then, for each drone $\delta \in \Delta_k$, we compute the additional lateness (if any) that the trip schedule $\Gamma^*_{k,j=\delta}$ would have in case drone δ is still flying. In this case, the time required by δ to complete its trip and return to the depot must also be taken into account. Moreover, we also consider the time required for the drone to have its battery swapped or the waiting time for a fully recharged one. Let $\psi_{\Gamma^*_{k,j=\delta}}$ denote the total lateness of the trip schedule $\Gamma^*_{k,j=\delta}$. This information, in combination with the trip schedules, is employed to solve the following Trip Assignment Problem (TAP):

Algorithm 7: Assign trips (*assign_trips*).

input : Current epoch time t_k; set of trips Γ_k^*; set of drones Δ_k.
output: Assignment of trip schedules to drones $\{\Gamma_k(\delta) : \delta \in \Delta_k\}$.

for $j = 1, \ldots, |\Delta|$ **do in parallel**
  for each $\Gamma_{k,j}^* \in \Gamma_k^*$ **do** // find the best permutation of each trip schedule
    $\Gamma_{k,j}^* \leftarrow \bar{\Pi}(\Gamma_{k,j}^* t_k)$
    for each $\delta \in \Delta_k$ **do** // compute the the total lateness of each possible assignment
      $\psi_{\Gamma_{k,j=\delta}^*} \leftarrow compute\_delay(\Gamma_{k,j=\delta}^*)$
    end
  end
end
$\{\Gamma_k(\delta) : \delta \in \Delta_k\} \leftarrow \texttt{TAP}(\{\psi_{\Gamma_{k,j}^*} : \Gamma_{k,j}^* \in \Gamma_k^*, \delta \in \Delta\}, \Delta_k)$ // solve model (10a)–(10d)
return $\{\Gamma_k(\delta) : \delta \in \Delta_k\}$

$$\text{(TAP)} \ \min \sum_{j=1}^{|\Delta|} \sum_{\Gamma_{k,j}^* \in \Gamma_k^*} \sum_{\delta \in \Delta_k} \psi_{\Gamma_{k,j}^*} x_{\Gamma_{k,j}^* \delta} \tag{10a}$$

subject to

$$\sum_{\forall \delta \in \Delta_k} x_{\Gamma_{k,j}^* \delta} = 1, \qquad \Gamma_{k,j}^* \in \Gamma_k^*,\ j = 1, \ldots, n \tag{10b}$$

$$\sum_{j=1}^{n} \sum_{\Gamma_{k,j}^* \in \Gamma_k^*} x_{\Gamma_{k,j}^* \delta} = 1, \qquad \forall \delta \in \Delta_k \tag{10c}$$

$$x_{\Gamma_{k,j}^* \delta} \in \{0, 1\}, \qquad \Gamma_{k,j}^* \in \Gamma_k^*,\ j = 1, \ldots, n,\ \forall \delta \in \Delta_k. \tag{10d}$$

where decision variables $x_{\Gamma_{k,j}^* \delta}$ are defined as follows:

$$x_{\Gamma_{k,j}^* \delta} = \begin{cases} 1, & \text{if the permutated trip schedule } \Gamma_{k,j}^* \text{ is assigned to drone } \delta \in \Delta_k, \\ 0, & \text{otherwise.} \end{cases}$$

Objective function (10a) seeks to minimize the overall lateness associated with the assignment of each permutation of trips to a drone in Δk. Constraints (10b) indicate that a drone is assigned to exactly a trip schedule, while constraints (10c) ensure that each trip schedule is assigned to a single drone. Variables $x_{\Gamma_{k,j}^* \delta}$ are binary, as defined in (10d), and define the action to be taken at epoch k. We observe that the solution provided by the TAP can assign a trip schedule to a drone where some trips can start at a time greater than or equal to $t_{k+1} = t_k + \Psi$. Using our CFA policy, these trips will be canceled in the next pre-decision state s_{k+1}, as their customer requests can be reassigned and rescheduled more effectively in the next epochs, given that new requests appear. On the contrary, if the myopic policy $\pi^{CFA}(M = +\infty)$ is used, all the trips assigned to a drone are executed and remain unchanged, even if they start in the next epochs.

6.7. Battery assignment procedure

After finishing a trip and returning to the depot, each drone is loaded with the deliveries to be performed on the next trip. It also undergoes a battery swap, replacing its depleted battery with a fully charged one. To ensure that the batteries are utilized uniformly and thus age in a homogeneous manner, it is necessary to monitor the number of times each battery β is employed. This is represented by ω_k^β. The batteries are sorted in a priority queue based on their usage count, with those used fewer times having higher priority. Therefore, upon the arrival of a drone at the depot, the first battery of this queue is removed and equipped to the drone. The battery swap is conducted within the drone finite-state machine, as illustrated in Fig. 1.

Once a recharged battery is equipped on a drone, the former drone's battery is removed and not inserted into the priority queue immediately. Instead, the battery is first recharged. The time to fully recharge the battery depends on its remaining charge and the recharge ratio, expressed as $e'_\beta = 5\,\%/min$. Consequently, an empty battery requires 20 minutes to recharge. Upon completion of the recharging process, the battery becomes available to be equipped on a drone again and inserted into the priority queue. If this queue is empty when a drone arrives at the depot, the drone waits for a battery to finish its recharge, thus becoming available for use.

6.8. Handling swappable batteries shortage

To ensure that our algorithm can address settings where the number of swappable batteries (if any) is less than the number of drones, we have implemented an additional step in our approach, alongside the priority queue described in Section 6.7. A swappable

battery is an extra battery, that is, an additional battery in addition to the ones of each drone. In the scenario of a shortage of swappable batteries, we can choose not to allocate all the available batteries in a given state. Instead, we maintain a percentage of them without swapping to anticipate a high realization of requests in a future epoch, in which many drones will have to leave to serve urgent requests.

The reserved batteries are only used if the number of urgent customers exceeds a certain threshold. In our tests, we reserve 10 % of the swappable batteries and use them only if the proportion of urgent customers exceeds 10 % of the outstanding customers. When there is no swappable battery, the drone waits for its battery to be charged, and it is dispatched as soon as the newly recharged battery is equipped.

7. A policy function approximation method

To benchmark the quality of solutions from our CFA approach, we consider a PFA policy, which defines the criteria for deciding (i) which drones stationed at the depot should have their batteries swapped with fully recharged ones; and (ii) which customer requests should be fulfilled based on urgency only, selecting the most urgent ones according to the $l_p - t_k$ values.

Let t_k be the time of state $s_k = (t_k, \mathcal{D}_k, \mathcal{U}_k, \Delta_k, \mathcal{B}_k)$. Here, the action involves arranging trips according to a predetermined value, R^U, representing the maximum number of urgent requests in $\mathcal{D}_k \cup \mathcal{U}_k$ that can be fulfilled in a single trip, regardless of distance minimization. It should be noted that the decision is made based on the threshold R^U, which is calibrated through simulation on a set of benchmark instances. This threshold remains constant for each state of the system. Specifically, when the system is at state s_k, a feasible action $x^{PFA} \in \mathcal{X}(s_k)$ is taken according to the following rules:

- the procedures described in Sections 6.1 and 6.2 are used to construct and update an initial set of trips using the backward dynamic programming Algorithm 3 and the trip update Algorithm 5, respectively. In these algorithms, set $\mathcal{V}$ accounts only for the most R^U urgent requests in $\mathcal{D}_k \cup \mathcal{U}_k$, while set $\Gamma_{(s_{k-1}, s_k)}$ is updated by considering any subset of requests in $\mathcal{D}_k \cup \mathcal{U}_k$ that can be inserted into some trips of $\Gamma(s_{k-1}, s_k)$. Let Γ_k be the set of trips returned by the two previous algorithms. If an urgent request appears in more than one trip $\gamma \in \Gamma_k$, it is only kept in the trip that requires the least energy. Let $\Gamma_k^* \subseteq \Gamma_k$ be the set of trips without repeated requests. The trip assignment procedure described in Section 6.6 is then executed to obtain a feasible multi-trip route plan for all drones $\delta \in \Delta_k$;
- if there is at least one drone at the depot, the available swappable batteries are assigned to these drones by distinguishing between two cases: *a*) the number of swappable batteries is equal to or greater than the number of drones; *b*) the number of swappable batteries is less than the number of drones. In the first case, battery assignment is performed according to the procedure described in Section 6.7. In the second case, the assignment follows the criterion outlined in Section 6.8.

The rules governing the structuring of actions in the PFA adhere to the principle of planning trips that visit a limited number of locations, while ensuring an adequate service level without incurring significant penalties. This approach also helps prevent critical situations where new requests appear in the system and there are insufficient drones available at the depot to fulfill them. Moreover, this policy differs from the CFA, which enables trips to be constructed regardless of the number of visited locations, including non-urgent ones.

8. Computational experiments

In this section, we assess the performance of our solution methodology on instances adapted from the literature. All algorithms have been implemented in C++ and compiled with the g++ compiler, version 12.3.0. We used Gurobi's C++ API v.12.0.2 to solve integer programs. All tests were executed on a computer with an Intel® Core™ i9-13900K processor with 32 threads at 3.0 GHz and 128 GB of RAM.

We generated our instances from those of Ulmer and Thomas (2018), which we obtained after requesting them from the first author. They shared 400 instances with 500 customers, of which 200 were generated using a uniform distribution and 200 using a normal distribution on the customers' coordinates. From each set of 200 instances, we randomly selected 50 and adapted them to be used in this work.

Ulmer and Thomas (2018) designed these instances to consider deliveries by drones and non-drone vehicles (e.g., trucks), whereas in our case, we only consider drones. Thus, we reduced the distances between all nodes by 15 %. After scaling the distances, we removed all unreachable customers, that is, customers whom a drone cannot serve, even when performing a round trip, because the energy required to reach it exceeds the battery capacity. Subsequently, customers were sampled randomly, and three new instances were derived with $m \in \{200, 300, 400\}$, resulting in 300 instances. In these instances, each customer node r is associated with a random demand $q_r \in [0.225kg, 1.5kg]$, a service time $\eta = 3$ min, a time a_r at which customer r appears in the system, and the customer's soft deadline l_r. The value a_r was randomly generated in the interval $(0, T - 240\,\text{min}]$, and we set $l_r = a_r + 240$ min following Ulmer and Thomas (2018). Furthermore, the instances with $m = 200$ were solved with $n = 12$ drones, those with $m = 300$ were solved using $n = 18$, and those with $m = 400$ were solved using $n = 24$. These instances and detailed results are available from https://github.com/Pigzaum/DRPUDEC.

Except for parameters M and Ψ, whose values may vary depending on the specific tests employed, all remaining parameters are based on the values outlined in Table 2, obtained from the literature or after a preliminary testing phase. Some values pertaining to drones and batteries were derived from the data presented in Dorling et al. (2017) and Ulmer and Thomas (2018).

Table 2
Parameter values.

												Drone data						Battery data				
Parameter	$\sigma_{\bar{\xi}}$	α	Σ	μ^{pr}	μ^{npr}	μ^{d}	μ^{l}	g	$\bar{l}$	v	$\bar{\xi}$	v	Q	m	p	z	b	ρ	e'_{β}	$E_{\min}$	$E_{\max}$
Value	20 %	90 %	5	0.8	0.2	1	5	$9.81 N/kg$	60 min	5 min	$24 km/h$	$3kg$	$2.3kg$	6	$1.204 kg/m^3$	$0.0064 m^2$	$1.5n$	20 min	5 %/min	10 %	100 %

In Table 2, $\sigma_{\bar{\xi}}$ is the standard deviation of the drone's average speed as a percentage of the average speed and α is the confidence interval, both used in inequality (7) to compute the drone's maximum energy consumption. We have used $\sigma_{\bar{\xi}} = 20\,\%$ as this value corresponds to a realistic setting (Tamke and Buscher, 2023). Moreover, in Table 2, Σ is the maximum number of adjacent nodes to be inserted in trip used in Algorithm 3; and μ^{pr} and μ^{npr} are the objective coefficients used in model (8a)–(8h). The values of parameters μ^{pr} and μ^{npr} were defined empirically. All other parameters are described in Table 1.

Section 8.1 presents preliminary tests to assess the steps of our solution approach. Section 8.2 reports the results obtained on the benchmark instances, comparing our CFA method with a myopic, a PFA, and an oracle method.

8.1. Preliminary tests

This section presents the results of preliminary tests conducted to evaluate the performance of the proposed algorithm. In these preliminary tests, 12 instances were used, with two instances of each value of $m \in \{200, 300, 400\}$ selected for both normal and uniform distributions. To choose these 12 instances, we picked the first two instances of each size and each distribution. Each instance was executed 10 times, and the results presented are the averages of these runs.

This section is organized as follows: the outcomes of the tests conducted to identify an optimal value for parameter M are presented in Section 8.1.1, and Section 8.1.2 outlines the computational analysis performed to evaluate each of the main steps of our solution approach. In addition, Section 8.1.3 presents tests evaluating the robustness of using chance constraints in our trip-building step to avoid premature returns of drones to the depot. Section 8.1.4 reports tests aimed at determining a suitable number of swappable batteries at the charging station.

8.1.1. Tunning parameter M

Two values of Ψ were considered: 30 min and 60 min. These values were fixed by varying $M \in \{1, 2, 3, 4\}$ and selecting the best values for the tests presented in Section 8.2. Fig. 2 presents the average number of customers served (m^s_{avg}) obtained by our CFA policy for each value of $M = 1, 2, 3, 4$ with $\Psi = 30$ min and $\Psi = 60$ min settings. Following the same structure, Fig. 3 shows the results for the average total lateness (ψ_{avg}).

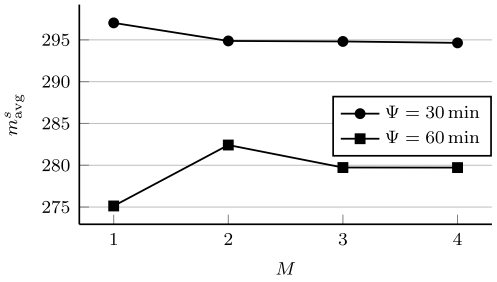

Fig. 2. Average number of served customers by our π^{CFA} over different M values for $\Psi = 30$ min and $\Psi = 60$ min.

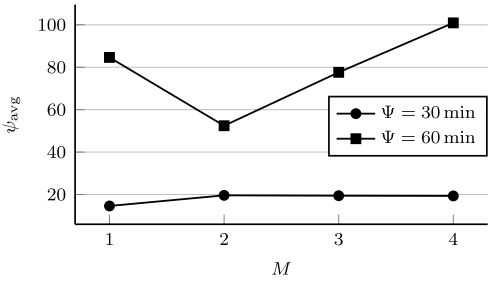

Fig. 3. Average total lateness of solutions obtained by our π^{CFA} over different M values for $\Psi = 30$ min and $\Psi = 60$ min.

As we can observe from the results presented in Figs. 2 and 3, $\pi^{CFA}(M = 1)$ yields the best values of m^s_{avg} and ψ_{avg} when $\Psi = 30$ min. Regarding the tests with $\Psi = 60$ min, the configuration with $M = 2$ yields the best results in terms of both the average number of customers served and the average total lateness. In consideration of these outcomes, we have selected $M = 1$ and $M = 2$, respectively for $\Psi = 30$ min and $\Psi = 60$ min, to be employed in the tests conducted in Section 8.2. These values of Ψ are consistent with the flight duration of a drone engaged in the delivery of goods in a practical setting.

8.1.2. Evaluating action components

In this section, we evaluate the main steps of our algorithm. First, we analyze the Trip Construction Heuristic by assessing the two insertion policies presented in Section 6.1, namely "by distance" and "by urgency" represented by sets $\mathcal{I}^d(.)$ and $\mathcal{I}^l(.)$, respectively. Second, we evaluate the trip selection step by testing Algorithm 6 in comparison to an alternative version that excludes the SPBP procedure, instead solely addressing the DTSSP. To demonstrate the importance of Algorithm 7, we evaluated a variant of our approach that omitted this phase. In this variant, the trip assignments to drones were conducted following the DTSSP solution. We employed $M = 1$ and $\Psi = 30$ min in all tests conducted in this section.

Table 3
Comparison of $\pi^{CFA}(M = 1)$ with different backward heuristic insertion policies.

	By distance						By urgency					
m	$\lvert\Gamma\rvert_{\text{avg}}$	ψ_{avg}	$\widetilde{CE}_{\text{avg}}$	$\text{H}_{t(s)}$	$\text{SPBP}_{t(s)}$	t(s)	$\lvert\Gamma\rvert_{\text{avg}}$	ψ_{avg}	$\widetilde{CE}_{\text{avg}}$	$\text{H}_{\text{t(s)}}$	$\text{SPBP}_{\text{t(s)}}$	t(s)
200	998.88	17.43	33.19	0.10	0.50	1.35	757.95	9.00	31.81	0.03	0.46	1.21
300	1609.53	53.13	50.46	0.22	2.06	4.51	1105.95	14.58	47.48	0.06	1.86	4.06
400	2048.90	19.68	66.46	0.37	5.26	10.57	1443.20	20.23	63.13	0.09	5.17	9.64
average	1552.43	30.08	50.04	0.23	2.61	5.48	1102.37	14.60	47.47	0.06	2.50	4.97

Table 4
Comparison of $\pi^{CFA}(M = 1)$ using different trips selection strategies.

	DTSSP				SPBP + DTSSP			
m	m^s_{avg}	ψ_{avg}	$\text{DTSSP}_{\text{t(s)}}$	t(s)	m^s_{avg}	ψ_{avg}	$\text{DTSSP}_{\text{t(s)}}$	t(s)
200	85.80	21.65	1.86	2.12	198.55	9.00	0.63	1.21
300	164.43	34.90	3.88	4.28	298.95	14.58	2.02	4.06
400	245.30	91.23	7.46	10.14	393.60	20.23	4.24	9.64
average	165.18	49.26	4.40	5.51	297.03	14.60	2.30	4.97

Table 3 compares the two insertion policies (by distance and by urgency) for building trips in the backward heuristic. In this table, we present the average results over the instances of the same size $m \in \{200, 300, 400\}$. For both policies, Table 3 shows the total number of generated trips ($\lvert\Gamma\rvert_{\text{avg}}$), the total lateness in minutes (ψ_{avg}), and the total energy consumption in kilowatts-minutes ($\widetilde{CE}_{\text{avg}}$). Additionally, the total time spent in the heuristic generating trips through all epochs ($\text{H}_{\text{t(s)}}$) is provided, as is the total execution time of solving the set-partitioning problem across all epochs ($\text{SPBP}_{\text{t(s)}}$). Finally, the total running time of the whole execution (t(s)) is also included.

As we can observe from Table 3, using the urgency insertion policy reduces the total number of trips generated, compared to the "by distance" one. This is because, in the "by urgency" approach, the algorithm can serve customers faster, i.e., in fewer epochs, and thus, there are some epochs in which no trips need be generated. The average number of trips generated (when an epoch requires them) is thus larger, but this happens less often than in the policy driven by distance.

As expected, the utilization of the urgency policy in the construction of trips reduces overall lateness compared to the "by distance" one. In addition, the average energy consumption per drone is less than that observed when optimizing distance. This is because, in the latter case, drones will be more heavily loaded and will, therefore, utilize their batteries to a greater extent. Consequently, the urgency policy demonstrates a reduction in energy consumption compared to the distance policy.

Regarding the running times, both versions and the whole algorithm are fast, but the backward heuristic is observed to run quicker when utilizing the urgency policy. This phenomenon can be attributed to how customer data is stored. Upon becoming visible to the system, a customer is stored in a priority queue in which outstanding customers are sorted by an increasing deadline. Consequently, customers are already sorted according to urgency. On the other hand, utilizing the "by distance" insertion policy necessitates periodically sorting outstanding customers by distance. This explains the difference in the running time of the backward heuristic. Furthermore, the implementation of the urgency policy results in a reduction in the number of trips generated, which in turn leads to a decrease in the execution time for solving the SPBP across all epochs in comparison to the distance-based policy. The implementation of the urgency insertion policy results in a reduction in the time required for the trip-building and selection steps. Accordingly, this insertion policy was selected for implementation in all tests conducted in this work.

Table 4 shows comparative tests between the two route selection strategies: first, solving only the DTSSP to select trips from the entire set generated, and second, solving SPBP and then the DTSSP, as presented in Algorithm 6. To test this algorithm without the SPBP step, we impose that all urgent customers must be visited at least once in the DTSSP instead of imposing that all customers visited in the SPBP solution must be visited in the DTSSP solution, as the SPBP solution is not generated in this case. Additionally, for the DTSSP-only configuration, we considered customers as urgent if their due time lies in $[t_k, t_k + 3\bar{l}]$. The objective of these tests was to demonstrate the significance of Algorithm 6 in conjunction with both the SPBP and DTSSP. Besides the total lateness and the total execution time of our method, Table 4 presents the number of customers served (m^s_{avg}) and the total execution time to solve the DTSSP through all epochs in seconds ($\text{DTSSP}_{\text{t(s)}}$). As in Tables 3, 4 presents average results over $m \in \{200, 300, 400\}$.

Table 4 demonstrates that configuring Algorithm 6 with the resolution of the SPBP and subsequently the DTSSP, in contrast to merely using the DTSSP is more efficient. The former approach results in solutions with significantly more customers visited and reduced total lateness, as better trips can be selected using the SPBP + DTSSP configuration. Moreover, it can be observed that employing the SPBP to select trips before the DTSSP reduces the time required for the latter to be solved. This is achieved by ensuring that customers included in trips selected by the SPBP are visited in a solution of the DTSSP, thereby producing a tighter problem. For these reasons, the configuration of Algorithm 6 with first solving the SPBP and then the DTSSP, has been employed in all tests conducted for this work.

Table 5 presents a series of tests designed to assess the efficacy of the TAP in assigning trips to drones. Two versions of the proposed method were tested: without and using Algorithm 7. In the first one, the algorithm assigns trips to drones in the same order as in

Table 5
Comparison of $\pi^{CFA}(M = 1)$ using or not Algorithm 7 to assign trips to drones.

	Without Algorithm 7			With Algorithm 7		
m	m^s_{avg}	ψ_{avg}	t(s)	m^s_{avg}	ψ_{avg}	t(s)
200	196.98	19.40	1.77	198.55	9.00	1.21
300	298.23	15.68	5.49	298.95	14.58	4.06
400	390.93	21.15	13.62	393.60	20.23	9.64
average	295.38	18.74	6.96	297.03	14.60	4.97

Table 6
Trips failure rates of $\pi^{CFA}(M = 1)$ by instance size.

	$\Psi = 30$ min			$\Psi = 60$ min		
m	# executed	# failed	Failure rate (%)	# executed	# failed	Failure rate (%)
200	121.00	0.60	0.50	121.45	0.60	0.49
300	189.95	0.48	0.25	189.35	0.63	0.33
400	253.95	0.68	0.27	256.73	1.55	0.60
average	188.30	0.58	0.31	189.18	0.93	0.49

the solution found by the DTSSP. In the second one, we solve the TAP to optimally assign trips to drones. Table 5 presents a structure similar to that observed in the preceding tables.

As demonstrated in Table 5, the application of the TAP following the selection of trips is another key aspect of our proposed algorithm. The application of the TAP results in enhanced overall efficiency, characterized by reduced lateness and an increased number of customers served. Furthermore, due to the optimization of trip-drone assignments and the enhanced efficiency of customer service, the overall running time of the algorithm is reduced. In light of these observations, the proposed algorithmic approach, as outlined in Algorithm 7, is employed in all tests presented in this work.

8.1.3. Operational robustness and premature returns

In addition to performance metrics such as service level and customer lateness, we also tracked the frequency of premature returns caused by unexpected energy consumption levels. The failure rates are shown in Table 6. In this table, column "# executed" presents the average number of routes executed by drones, column "# failed" is the average number of trips executed that have failed, and column "Failure rate (%)" shows the failure rate.

The frequency of such premature returns was found to be below 0.6 % in both the $\Psi = 30$ min and $\Psi = 60$ min settings, indicating that the chance constraints effectively mitigate the risk of in-flight energy depletion. Specifically, the effectiveness of the policy in preventing premature returns to the depot is primarily due to the parameter $\alpha = 90\,\%$ in the chance constraints.

Furthermore, our model includes mechanisms to detect potential energy shortfalls during flight. If a drone's projected energy consumption exceeds the available battery capacity minus E_{min}, the drone is instructed to return to the depot immediately. This proactive strategy ensures that drones do not attempt to complete deliveries when there is a significant risk of battery depletion.

8.1.4. Setting the number of swappable batteries

In this section, we present the results of the tests performed to assess the performance of $\pi^{CFA}(M = 1)$, with $\Psi = 30$ min, varying the number of swappable batteries. Our objective with these tests is to determine an optimal number of swappable batteries at the charging station, aiming to serve the maximum number of customers on average while minimizing average lateness. Specifically, we tested $b \in \{n, 1.1n, 1.25n, 1.5n, 1.75n, 2n\}$, representing scenarios from zero swappable batteries ($b = n$) to one swappable battery per drone ($b = 2n$). These results are summarized in Figs. 4 and 5: the former reports the average number of customers served for each battery configuration, while the latter shows the corresponding average lateness.

As shown in Figs. 4 and 5, the number of batteries plays a central role in the quality of service provided by the drones. There is a significant improvement in both the number of customers served and lateness when moving from having no extra battery ($b = n$) to having one additional battery for every two drones ($b = 1.5n$). However, the improvement from $b = 1.5n$ to $b = 2n$ is marginal. Moreover, it should be noted that even in the scenario where each drone has only its own battery ($b = n$), our $\pi^{CFA}(M = 1)$ method was still able to produce solutions serving, on average, more than 263 customers with an average lateness of at most 122 min, showing the effectiveness of the methods described in Sections 6.7 and 6.8. Based on these results and considering that a scenario with half as many extra batteries as drones may be more realistic, we decided to use $b = 1.5n$ in all the remaining tests in this work.

8.2. Numerical results

In this section, we present the tests performed with all 300 benchmark instances we have derived from those of Ulmer and Thomas (2018). In Section 8.2.1, we compare our π^{CFA} with a myopic method. Section 8.2.2 shows the results of π^{CFA} against a PFA method, while Section 8.2.3 assesses the performance of π^{CFA} compared with an oracle method.

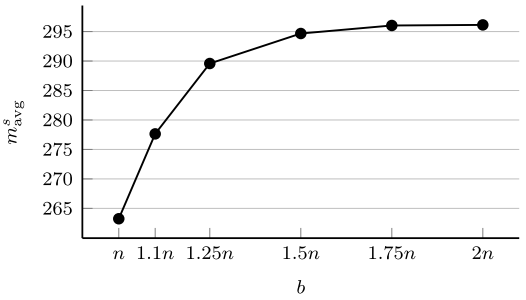

Fig. 4. Average number of served customers by $\pi^{CFA}(M = 1)$ over different b values for $\Psi = 30$ min.

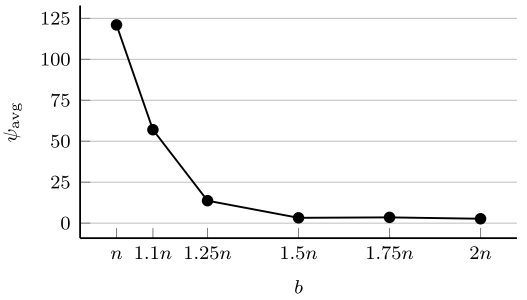

Fig. 5. Average lateness by $\pi^{CFA}(M = 1)$ over different b values for $\Psi = 30$ min.

8.2.1. Comparison against a myopic approach

We compare our method using two settings: $\pi^{CFA}(M = +\infty)$, that is, a myopic approach where trips are not canceled and it does not limit the number of trips assigned to drones; and $\pi^{CFA}(M = M^*)$, where we have used $M^* = 1$ for $\Psi = 30$ min and $M^* = 2$ for $\Psi = 60$ min, calibrated after a simulation procedure described earlier.

The box plots depicted in Figs. 6 and 7 provide a statistical overview of the performance of the $\pi^{CFA}(M = 1)$ in comparison to the myopic one when $\Psi = 30$ min. Fig. 6 displays box plots of the pairwise percent increase on the average number of customers served. Likewise, Fig. 7 displays box plots of the pairwise percent reduction in average cost per served customer.

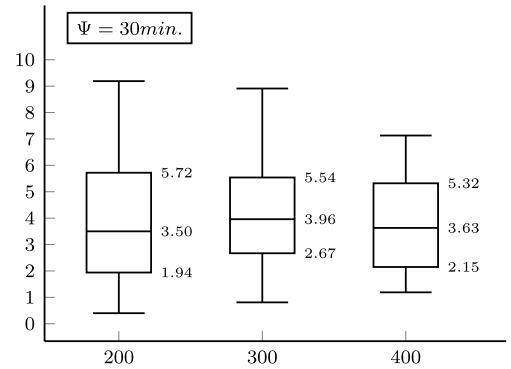

Fig. 6. Percent increase in average number of served customers by $\pi^{CFA}(M = 1)$ versus $\pi^{CFA}(M = +\infty)$.

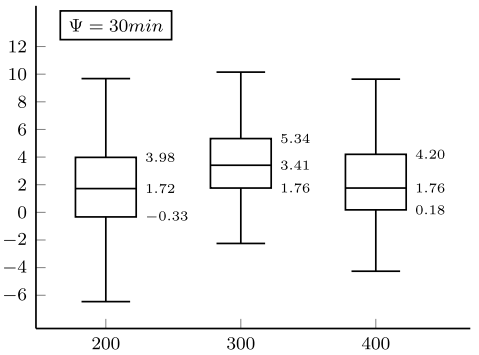

Fig. 7. Percent reduction in average cost per served customer by $\pi^{CFA}(M = 1)$ versus $\pi^{CFA}(M = +\infty)$.

One can note from the box plot of Fig. 6 that for the instances with 200 customer demands, the CFA policy achieved a median percent increase of 3.50 %, 3.96 % in the instances with 300 customer demands, and 3.63 % in the instances with 400 customer demands. The 25th percentiles of the percent increase values are positive for all the data sets, demonstrating that the CFA policy consistently achieves a greater total number of served customers in all instances.

Regarding the box plot of Fig. 7, for all the instances, we observe a variability in the average cost per served customer in the solutions provided by the CFA policy compared to those obtained with the myopic policy. The CFA policy achieves a median percent reduction of 1.72 % in the data set with 200 customer demands, 3.41 % in the data set with 300 customer demands, and 1.76 % in instances with 400 customer demands.

Table 7 shows a breakdown of the results of experiments conducted with $\Psi = 30$ min. In this table, m^s_{avg} denotes the average number of customers served, $m^{\text{swl}}_{\text{avg}}$ the average number of customers served without delay, and c_{avg} is the average solution cost, which is composed by the average total lateness (ψ_{avg}) and by the average total distance traveled (τ_{avg}). In addition, column t(s) shows the algorithm's execution time in seconds.

We can observe that the CFA policy with $M = 1$ dominates the myopic policy, which corresponds to $M = +\infty$. For all sizes of instances, the number of customers served is larger in our calibrated CFA policy. In particular, in the dataset with 200 customers, the $\pi^{CFA}(M = 1)$ policy served, on average, 7.48 more customers without lateness compared to the myopic policy. Moreover, the average lateness has a gap that is 17.39 % lower in the CFA policy compared to the myopic approach. In the dataset with 300 customer demands, even though it served, on average, 12.12 more requests than the myopic policy, the $\pi^{CFA}(M = 1)$ method achieved a slightly better solution cost when compared to the myopic method. In addition, the average lateness decreased by 39.63 % in comparison to the myopic policy. Finally, in the dataset with 400 customer demands, the $\pi^{CFA}(M = 1)$ method served, on average, 14.55 more customers without lateness compared to the myopic approach, while the average lateness shows a decrease of 11.14 % in the gap compared to that observed in the myopic policy.

Table 7
Average results of tests with $\Psi = 30$ min.

	$\pi^{CFA}(M=+\infty)$						$\pi^{CFA}(M=1)$					
m	m^{s}_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)	m^{s}_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)
200	189.62	189.19	858.57	7.30	822.05	3.29	196.93	196.67	866.74	6.03	836.58	1.15
300	284.00	283.32	1280.71	10.22	1229.60	24.80	295.73	295.44	1280.55	6.17	1249.71	3.90
400	379.78	379.06	1692.50	10.50	1639.99	58.15	394.06	393.61	1711.92	9.33	1665.27	9.76
average	284.46	283.86	1277.26	9.34	1230.55	28.75	295.57	295.24	1286.41	7.18	1250.52	4.93

Table 8
Average results of tests with $\Psi = 60$ min.

	$\pi^{CFA}(M=+\infty)$						$\pi^{CFA}(M=2)$					
m	m^{s}_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)	m^{s}_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)
200	181.23	179.74	917.36	26.76	783.55	1.85	188.05	187.72	830.49	5.86	801.18	1.24
300	270.42	268.31	1360.57	38.82	1166.46	10.31	281.37	280.97	1236.10	7.84	1196.89	6.08
400	358.94	355.98	1848.77	59.39	1551.81	40.14	373.90	373.33	1650.37	10.63	1597.21	20.74
average	270.20	268.01	1375.57	41.66	1167.27	17.43	281.11	280.67	1238.99	8.11	1198.43	9.35

Figs. 8 and 9 show a statistical overview of the performance of the CFA policy in comparison to the myopic one using $\Psi = 60$ min. Fig. 8 depicts box plots of the pairwise percent increase in average number of served customers achieved by $\pi^{CFA}(M=2)$ versus $\pi^{CFA}(M=+\infty)$. Similarly, Fig. 9 shows the equivalent information for the reduction in average cost per served customer.

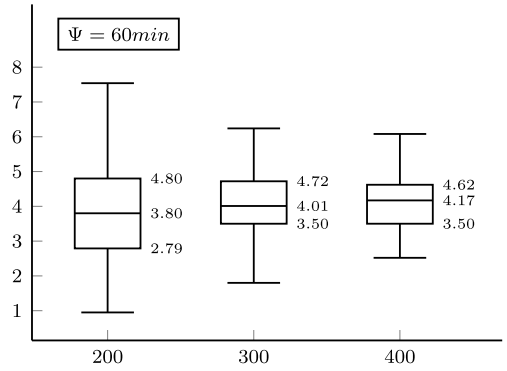

Fig. 8. Percent increase in average number of served customers by $\pi^{CFA}(M=2)$ versus $\pi^{CFA}(M=+\infty)$.

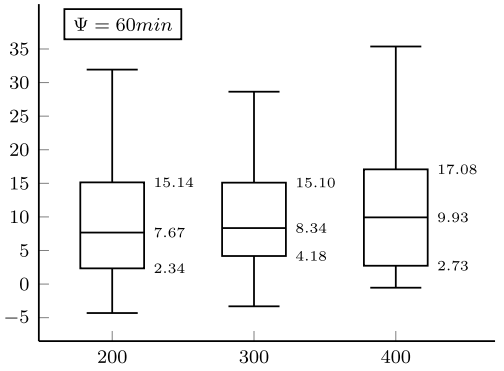

Fig. 9. Percent reduction in average cost per served customer by $\pi^{CFA}(M=2)$ versus $\pi^{CFA}(M=+\infty)$.

As it can be noted from Fig. 8, for the instances with 200 customers, the CFA policy achieves a median percentage increase of 3.80 %, of 4.01 % for instances with 300 customers, and 4.17 % for those with 400 customers. The 25th percentiles of the percent increase values are positive for all the data sets, indicating that the CFA policy consistently results in a greater total number of served customers.

Considering results presented in the box plot of Fig. 9, for all the instances, we observe a reduction in the average cost per served customer in the solutions provided by the CFA policy compared to those obtained with the myopic policy. The CFA policy achieves a median percent reduction of 7.67 % in the data set with 200 customer demands, 8.34 % in the data set with 300 customer demands, and 9.93 % in instances with 400 customer demands. In contrast to the case where $\Psi = 30$ min, we observe an increasing trend for the median percentage reduction in cost per served customer.

Table 8 presents the detailed average results using $\Psi = 60$ min. This table follows the same structure as Table 7. The $\pi^{CFA}(M=2)$ policy consistently obtained solutions in which more customers were served compared to the myopic policy. Moreover, even while serving more customers, the $\pi^{CFA}(M=2)$ policy found solutions with lower cost than those of the myopic approach. Specifically, the data set with 200 customer demands shows a reduction in the gap of 9.47 % and 78.10 % for the average total cost and average lateness, respectively. For the data set with 300 customer demands, we observe a reduction in the average total cost and average lateness, with decreases of 9.15 % and 79.80 %, respectively. In the data set with 400 customer demands, we observe a decrease of 10.73 % in the gap between the average total cost for the CFA policy and the myopic policy. In addition, we observe a decrease of 82.10 % in the average lateness gap.

The analysis of the average routing cost in columns τ_{avg} of Tables 7 and 8 shows that the CFA policy might seem to perform slightly worse than the myopic policy. There are two explanations for this observation. First, recall that the first objective of our algorithm is to serve more customers. Our CFA policy excels in this criterion. Second, the lack of service to all customers, as evidenced in columns

Table 9
Average results of PFA tests with $\Psi = 30$min.

	PFA						$\pi^{CFA}(M=1)$					
m	m^s_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)	m^s_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)
200	173.70	169.58	1116.55	84.60	693.57	0.34	196.93	196.67	866.74	6.03	836.58	1.15
300	251.20	238.86	2573.29	318.88	978.87	0.50	295.73	295.44	1280.55	6.17	1249.71	3.90
400	317.71	287.67	6691.32	1093.72	1222.71	0.68	394.06	393.61	1711.92	9.33	1665.27	9.76
average	247.53	232.04	3460.38	499.07	965.05	0.51	295.57	295.24	1286.41	7.18	1250.52	4.93

m^s_{avg}, can be attributed to the reduced flight range of the drones, which is influenced by the uncertainty in power consumption of the batteries and the limited number of drones available to complete all deliveries.

8.2.2. *Comparison against a policy function approximation*

In this section, we present the results of the PFA, described in Section 7, and compare them with the $\pi^{CFA}(M = M^*)$ methods. In the PFA, we consider urgent customers to be those whose deadlines fall within the interval $[t_k, t_k + 3\bar{l}]$, rather than the interval $[t_k, t_k + \bar{l}]$ used in the CFA policy. Thus, at each iteration of the PFA, the algorithm selects and executes trips that serve customers with deadlines in $[t_k, t_k + 3\bar{l}]$. This interval was determined empirically.

Fig. 10 shows the percentage increase in the average number of customers served by $\pi^{CFA}(M = 1)$ compared to the PFA. The percentage reduction in the average cost per served customer achieved by $\pi^{CFA}(M = 1)$ relative to the PFA is presented in Fig. 11. In both figures, we use $\Psi = 30$ min.

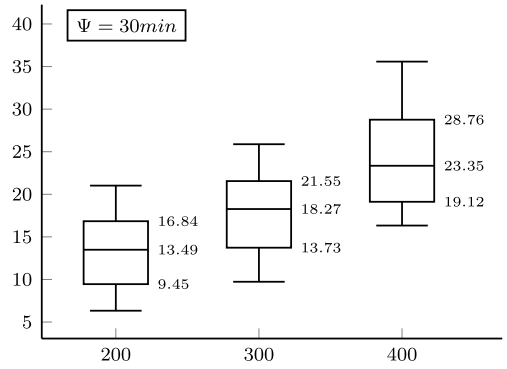

Fig. 10. Percent increase in average number of served customers by $\pi^{CFA}(M = 1)$ versus the PFA.

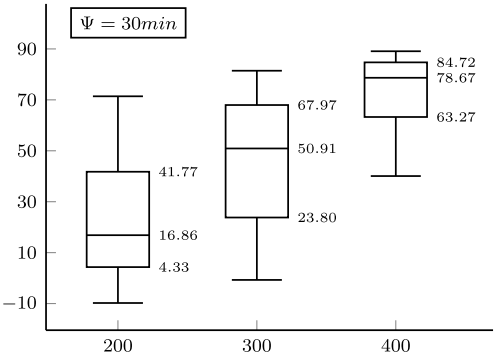

Fig. 11. Percent reduction in average cost per served customer by $\pi^{CFA}(M = 1)$ versus the PFA.

As shown in the box plots of Figs. 10 and 11, there is an increasing trend in the median values as the number of customers increases. Specifically, in Fig. 10, the $\pi^{CFA}(M = 1)$ policy achieved a median percentage increase of 13.49 % for instances with 200 customers, 18.27 % for 300 customers, and 23.35 % for 400 customers. In Fig. 11, the CFA method achieved a median percentage reduction in cost per served customer of 16.86 %, 50.91 %, and 78.67 % for 200, 300, and 400 customers, respectively. In addition, all 25th percentiles in the box plots of both figures are positive. These results clearly demonstrate that $\pi^{CFA}(M = 1)$ performed statistically better than the PFA, both in terms of the number of customers served and the cost per served customer.

Tables 9 and 10 present the detailed average results of the PFA; these tables follow the same structure of the previous ones. For completeness, the results of $\pi^{CFA}(M = 1)$ and $\pi^{CFA}(M = 2)$ from Tables 7 and 8 have been included in Tables 9 and 10, respectively.

As one can note from the results of Table 9, the $\pi^{CFA}(M = 1)$ dominates the PFA in almost every metric presented. The only results where the CFA did not perform better than the PFA are for τ_{avg} and the execution time. For the average total distance traveled by the drones, the reason is the same as in the results presented in the previous section: as the drones visit fewer customers in the PFA than in the CFA, they therefore fly less. Regarding the execution time, the PFA is faster than the $\pi^{CFA}(M = 1)$ because it does not use the mathematical models of Sections 6.4–6.6, which also explains why the CFA provide better quality results.

Figs. 12 and 13 are analogous to Figs. 10 and 11, but present the results of $\pi^{CFA}(M = 2)$ compared to the PFA. In this case, we have used $\Psi = 60$ min. These figures also show a statistically significant performance improvement of $\pi^{CFA}(M = 2)$ over the PFA policy. In both figures, all median values exceed 35.36 %, and all 25th percentiles are positive.

The results presented in Table 10 are similar to the ones presented in Table 9. As before, the $\pi^{CFA}(M = 2)$ policy outperforms the PFA in most metrics, with the exceptions again being the average total distance traveled and the execution time.

Note that the performance of the PFA worsens when $\bar{l} = 60$ min compared with $\bar{l} = 30$ min. This occurs because the PFA serves only urgent customers, i.e., those whose deadlines lie in $[t_k, t_k + \bar{l}]$, where $\bar{l} \in \{30\,\text{min}, 60\,\text{min}\}$. When $\bar{l} = 30$ min, information is updated more frequently, and fewer non-urgent customers accumulate over the epochs (note that the non-urgent customers of an epoch t_k will eventually become urgent in an epoch $t_{k'}$, with $k' > k$). In contrast, when $\bar{l} = 60$ min, although more customers are classified as urgent

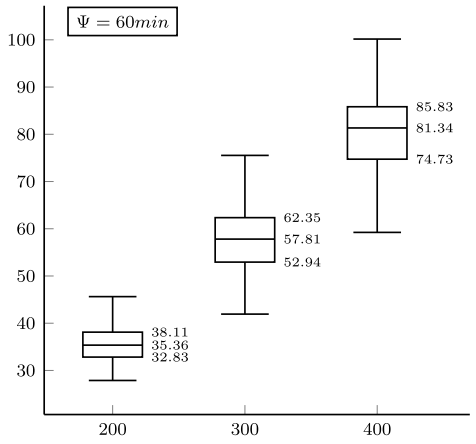

Fig. 12. Percent increase in average number of served customers by $\pi^{CFA}(M = 2)$ versus the PFA.

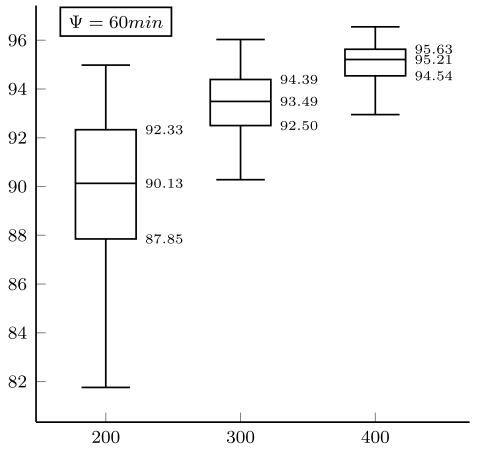

Fig. 13. Percent reduction in average cost per served customer by $\pi^{CFA}(M = 2)$ versus the PFA.

Table 10
Average results of PFA tests with $\Psi = 60$ min.

	PFA						$\pi^{CFA}(M = 2)$					
m	m^s_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)	m^s_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)
200	173.70	145.00	6568.41	1208.89	523.94	0.25	188.05	187.72	830.49	5.86	801.18	1.24
300	178.52	140.29	12287.54	2326.50	655.06	0.36	281.37	280.97	1236.10	7.84	1196.89	6.08
400	206.90	153.49	18993.38	3649.25	747.13	0.47	373.90	373.33	1650.37	10.63	1597.21	20.74
average	186.37	146.26	12616.44	2394.88	642.05	0.36	281.11	280.67	1238.99	8.11	1198.43	9.35

in each PFA update, the backlog of non-urgent customers becomes larger. In the final iterations, the number of urgent customers is very high, and the algorithm is unable to serve them all on time, or in some cases, at all. Consequently, the performance of the PFA worsens as $\bar{l}$ increases. On the other hand, the CFA gives priority to urgent customers but also tries to serve non-urgent ones at each epoch, which is why it outperforms the PFA.

8.2.3. Comparison against an oracle

In this section, we compare the results of our $\pi^{CFA}(M = 1)$ and $\pi^{CFA}(M = 2)$ policies, using $\Psi = 30$ min and $\Psi = 60$ min, respectively, against an oracle method. The oracle is an offline approach in which all requests are known before the start of the planning horizon. Moreover, drone speed and, thus, battery energy consumption, are deterministic - hence, there are no drone failures when executing the routes.

In the oracle method, Algorithms 3, 5–7 are executed with full knowledge of all requests throughout the time horizon. To improve the variability of the set of trips provided as input to Algorithms 6, 3 is run twice: once using the "urgency" insertion policy and once using the "nearest" insertion policy. As in the CFA method, we also include round trips for all customers. Comparing our $\pi^{CFA}(M = M^*)$ approach with the oracle provides a baseline, enabling us to assess the performance of our method in comparison to an ideal scenario.

Figs. 14 and 15 show, respectively, the average number of customers served and the average total lateness for the oracle method in comparison with $\pi^{CFA}(M = M^*)$. In these figures, due to the significant differences in the results, we use a logarithmic scale of base 2 to allow a better visualization of the results.

As shown in Fig. 14, the oracle consistently serves more customers than the $\pi^{CFA}(M = M^*)$ methods, as expected since the oracle has access to all information from the start of execution. However, this difference is not as large as one might assume, especially considering that new information about requests is revealed only every Ψ min in $\pi^{CFA}(M = M^*)$, and the non-determinism in energy consumption directly affects the results of these methods. In fact, for $\pi^{CFA}(M = 1)$, the difference is only marginal: $\pi^{CFA}(M = 1)$ served an average of 295.57 customers (see Table 7), compared to 298.17 for the oracle (see Table 11), which is only 0.87 % fewer customers than the oracle. A similar observation holds when comparing the oracle with $\pi^{CFA}(M = 2)$, where the latter served only 5.72 % fewer customers than the former. These results demonstrate the effectiveness of the CFA methods in producing solutions that serve nearly as many customers as an ideal scenario.

Considering the average lateness results shown in Fig. 15, the CFA methods significantly outperformed the oracle, finding solutions with much less lateness. This is due to two main factors affecting the oracle. First, Algorithm 3 is called only twice for all requests in the oracle (once using the "urgency" insertion policy and other using the "nearest" one), whereas in $\pi^{CFA}(M = M^*)$, it is called every Ψ min for a smaller set of customers. As a result, the diversification of trips is much greater in the CFA methods than in the oracle, which directly impacts the quality of the solutions in subsequent steps.

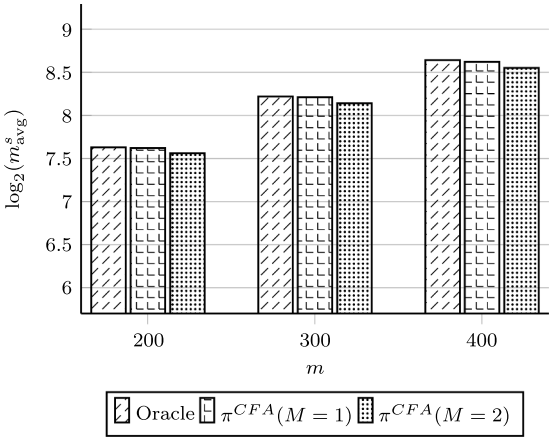

Fig. 14. Average number of customers served by the oracle, $\pi^{CFA}(M = 1)$, and $\pi^{CFA}(M = 2)$.

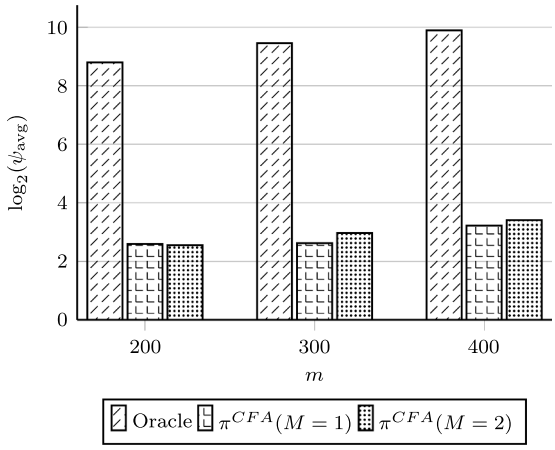

Fig. 15. Average lateness by the oracle, $\pi^{CFA}(M = 1)$, and $\pi^{CFA}(M = 2)$.

Table 11
Average results of the oracle method.

m	m^s_{avg}	m^{swl}_{avg}	c_{avg}	ψ_{avg}	τ_{avg}	t(s)
200	198.64	171.41	3106.60	445.84	877.40	310.96
300	298.25	256.18	4811.47	701.15	1305.72	315.29
400	397.63	339.70	6489.23	949.65	1740.98	328.35
average	298.17	255.76	4802.43	698.88	1308.03	318.20

The second reason for the oracle's poor performance in terms of lateness is related to Algorithm 7. In this algorithm, we search for the best permutation of each drone's trip schedule by enumerating all possible permutations and selecting the best one. Clearly, this is computationally expensive, as the number of possibilities grows factorially. In the CFA methods, the sets of trips are bounded by M, so this is not an issue since we use $M \leq 2$. However, in the oracle, this number can be as large as 18 trips. Therefore, we had to impose a time limit on Algorithm 7, which directly affects the quality of the average lateness results. In our tests, we used a five-minute time limit for Algorithm 7. These results also highlight the performance of $\pi^{CFA}(M = M^*)$, as it can find high-quality solutions in much shorter execution times.

Indeed, Table 11 presents the detailed average results of the oracle method in each set of instances. This table follows the same structure as the tables from previous sections. The average solution costs and the average lateness are significantly higher than those obtained by the $\pi^{CFA}(M = M^*)$ methods. In addition, because the enumeration step in Algorithm 7 is limited to five minutes, the total execution time of the oracle method is also significantly greater than that of our CFA policy.

9. Conclusion

This study considers the operational optimization problem of a fleet of drones deployed daily to make customer deliveries. The inherent uncertainty of this operation arises from the variability of customer demands and the drones' battery energy consumption. The objective is to design a system to plan trips and optimize their assignment to drones in a dynamic setting, considering the evolving delivery requirements throughout the day.

To develop our real-time policies to assign and schedule trips fulfilling the demands, we have designed a tailored CFA policy that accounts for a restricted number of trips to be assigned to the drones. This ensures that the drones are ready at the depot to fulfill new requests that may arise during the day. Our computational results demonstrate that the proposed CFA policy achieves lower-cost solutions than a myopic policy. Moreover, the CFA policy significantly reduces customer lateness compared to the myopic policy.

We have also developed a PFA policy and conducted a comparison against the CFA one. Our results show that the CFA policy consistently outperforms the PFA in terms of both the number of customers served and solution quality. In addition, we compared the CFA policy against an oracle method and found that the CFA performed very well, achieving similar results in terms of customers served while providing solutions with much less lateness, with execution times being only a fraction of those of the oracle.

Our model and solution approach can accommodate other problems in which uncertainty affects travel times. This includes, for instance, multi-modal transport systems that employ autonomous robots for delivery purposes. Further research is required to compare the proposed approach with different delivery forms in a dynamic context.

CRediT authorship contribution statement

Guilherme O. Chagas: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Leandro C. Coelho:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization; **Demetrio Laganà:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Patrizia Beraldi:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Conceptualization.

Data availability

We have shared the link to the instances and detailed results in the paper

Acknowledgement

This work was partly supported by the Canadian Natural Sciences and Engineering Research Council (NSERC) [grant number 2025–03964]. The work of Demetrio Laganá is partly supported by Ministero delle Imprese e del Made in Italy, Fondo per la Crescita Sostenibile - Accordi per l'innovazione di cui al D.M. 31 dicembre 2021 e D.D. 18 marzo 2022, project F/310044/01-04/X56 - Crawford "advanCed seRvices And netWorks FOr aiR Drone transport". Finally, we also thank the editorial board editor, the associate editor, and three anonymous referees for their valuable comments on an earlier version of this paper.

References

Bellman, R.E., Dreyfus, S., 1959. Functional approximations and dynamic programming. Math. Tables Other Aids Comput. 13, 247–251.
Cazenave, T., Lucas, J.-Y., Triboulet, T., Kim, H., 2021. Policy adaptation for vehicle routing. AI Commun. 34 (1), 21–35.
Chen, X., Ulmer, M.W., Thomas, B.W., 2022. Deep q-learning for same-day delivery with vehicles and drones. Eur. J. Oper. Res. 298 (3), 939–952.
Chen, X., Wang, T., Thomas, B.W., Ulmer, M.W., 2023. Same-day delivery with fair customer service. Eur. J. Oper. Res. 308 (2), 738–751.
Cheng, C., Adulyasak, Y., Rousseau, L.-M., 2020. Drone routing with energy function: formulation and exact algorithm. Transp. Res. Part B: Methodol. 139, 364–387.
Cheng, C., Adulyasak, Y., Rousseau, L.-M., 2024. Robust drone delivery with weather information. Manuf. Serv. Oper. Manage. 26 (4), 1402–1421.
Coelho, B.N., Coelho, V.N., Coelho, I.M., Ochi, L.S., Haghnazar, R.K., Zuidema, D., Lima, M. S.F., da Costa, A.R., 2017. A multi-objective green UAV routing problem. Comput. Oper. Res. 88, 306–315.
Dorling, K., Heinrichs, J., Messier, G.G., Magierowski, S., 2017. Vehicle routing problems for drone delivery. IEEE Trans. Syst. Man, Cybern.: Syst. 47 (1), 70–85.
Ermağan, U., Yıldız, B., Salman, F.S., 2022. A learning based algorithm for drone routing. Compu. Oper. Res. 137, 105524.
Faiz, T.I., Vogiatzis, C., Liu, J., Noor-E-Alam, M., 2024. A robust optimization framework for two-echelon vehicle and UAV routing for post-disaster humanitarian logistics operations. Networks 84 (2), 200–219.
French, S., 2020. How long do LiPo batteries last, and how do i know if an old LiPo battery is safe to use? Accessed: 2025-07-11. https://www.thedronegirl.com/2020/05/24/lipo-batteries-last/.
Gao, Y., Zhang, S., Zhang, Z., Zhao, Q., 2024. The stochastic share-a-ride problem with electric vehicles and customer priorities. Comput. Oper. Res. 164, 106550.
Garg, V., Niranjan, S., Prybutok, V., Pohlen, T., Gligor, D., 2023. Drones in last-mile delivery: a systematic review on efficiency, accessibility, and sustainability. Transp. Res. Part D: Transp. Environ. 123, 103831.
Macrina, G., Di Puglia Pugliese, L., Guerriero, F., Laporte, G., 2020. Drone-aided routing: a literature review. Transp. Res. Part C: Emerging Technol. 120, 102762.
Muli, C., Park, S., Liu, M., 2022. A comparative study on energy consumption models for drones. In: Lecture Notes in Computer Science: Global IoT Summit. Springer, pp. 199–210.
Murray, C.C., Chu, A.G., 2015. The flying sidekick traveling salesman problem: optimization of drone-assisted parcel delivery. Transp. Res. Part C: Emerging Technol. 54, 86–109.
Poikonen, S., Campbell, J.F., 2020. Future directions in drone routing research. Networks 77, 116–126.
Powell, W.B., 2011. Approximate Dynamic Programming: Solving the Curses of Dimensionality. Vol. 703 of *Wiley Series in Probability and Statistics*. John Wiley & Sons.
Powell, W.B., 2016. Perspectives of approximate dynamic programming. Ann. Oper. Res. 241, 319–356.
Prékopa, A., 1970. On probabilistic constrained programming. In: Proceedings of the Princeton Symposium on Mathematical Programming. Princeton University Press, Princeton, NJ, pp. 113–138.
Pugliese, L. D.P., Guerriero, F., Scutellá, M.G., 2021. The last-mile delivery process with trucks and drones under uncertain energy consumption. J. Optim. Theory Appl. 191, 31–67.
Ruszczyński, A., Shapiro, A., 2003. Stochastic Programming, Handbook in Operations Research and Management Science. Elsevier Science, Amsterdam.
Soeffker, N., Ulmer, M.W., Mattfeld, D.C., 2022. Stochastic dynamic vehicle routing in the light of prescriptive analytics: a review. Eur. J. Oper. Res. 298 (3), 801–820.
Stoia, S., Laganá, D., Ohlmann, J.W., 2025. Dynamic pickup-and-delivery for collaborative platforms with time-dependent travel and crowdshipping. Eur. J. Oper. Res. 322 (1), 70–84.
Tamke, F., Buscher, U., 2023. The vehicle routing problem with drones and drone speed selection. Comput. Oper. Res. 152, 106112.
Ulmer, M.W., 2017. Approximate Dynamic Programming for Dynamic Vehicle Routing. Vol. 61. Springer Cham.
Ulmer, M.W., Goodson, J.C., Mattfeld, D.C., Hennig, M., 2019. Offline-online approximate dynamic programming for dynamic vehicle routing with stochastic requests. Transp. Sci. 53 (1), 185–202.
Ulmer, M.W., Savelsbergh, M., 2020. Workforce scheduling in the era of crowdsourced delivery. Transp. Sci. 54 (4), 1113–1133.
Ulmer, M.W., Thomas, B.W., 2018. Same-day delivery with heterogeneous fleets of drones and vehicles. Networks 72 (4), 475–505.
Ulmer, M.W., Thomas, B.W., Campbell, A.M., Woyak, N., 2021. The restaurant meal delivery problem: dynamic pickup and delivery with deadlines and random ready times. Transp. Sci. 55 (1), 75–100.
van Steenbergen, R., Mes, M., van Heeswijk, W., 2023. Reinforcement learning for humanitarian relief distribution with trucks and UAVs under travel time uncertainty. Transp. Res. Part C: Emerging Technol. 157, 104401.
Yang, Y., Yan, C., Cao, Y., Roberti, R., 2023. Planning robust drone-truck delivery routes under road traffic uncertainty. Eur. J. Oper. Res. 309 (3), 1145–1160.
Yin, Y., Yang, Y., Yu, Y., Wang, D., Cheng, T., 2023. Robust vehicle routing with drones under uncertain demands and truck travel times in humanitarian logistics. Transp. Res. Part B: Methodol. 174, 102781.
Zhang, J., Campbell, J.F., Sweeney II, D.C., Hupman, A.C., 2021. Energy consumption models for delivery drones: a comparison and assessment. Transp. Res. Part D: Transp. Environ. 90, 102668.
Zhang, J., Woensel, T.V., 2023. Dynamic vehicle routing with random requests: a literature review. Int. J. Prod. Econ. 256, 108751.
Zhao, L., Bi, X., Li, G., Dong, Z., Xiao, N., Zhao, A., 2022. Robust traveling salesman problem with multiple drones: parcel delivery under uncertain navigation environments. Transp. Res. Part E: Logist. Transp. Rev. 168, 102967.